

本源司南 V4.0
用户手册与 API 参考文档

本源量子计算科技（合肥）股份有限公司



Contents

1 简介 **1**

1.1 手册目的 1

1.2 系统概述 1

1.3 术语与缩写 1

2 安装与部署 **3**

2.1 硬件要求 3

2.2 软件依赖 3

2.3 部署准备 3

2.3.1 解压部署包 4

2.3.2 安装部署依赖 4

2.4 部署流程 4

2.4.1 数据库初始化 4

2.4.2 修改配置文件 4

2.4.3 加载镜像文件 4

2.5 服务启动与验证 4

2.5.1 启动服务 4

2.5.2 验证访问 5

3 系统架构 **6**
3.1 整体架构图 6
3.2 执行流程 7
3.3 系统服务层核心模块 9
3.3.1 QCloudServer 模块 9
3.3.2 ComputerServer 模块 10
3.3.3 OSServerManager 模块 10
3.3.4 ScheduleServer 模块 11
3.3.5 QCompileServer 模块 11
3.3.6 ResourceManager 模块 12
3.3.7 BackendServer 模块 12

4 可视化管理控制平台 **14**
4.1 许可证验证 15
4.2 用户账号管理 16
4.2.1 用户注册 16
4.2.2 用户登录 18
4.2.3 用户管理 18
4.3 系统仪表盘概览 19
4.3.1 任务概览 19
4.3.2 资源使用情况 20
4.3.3 芯片任务耗时分布 20
4.3.4 文档访问 21
4.3.5 快捷操作 21
4.4 资源管理页面概览 21
4.4.1 芯片管理 21
4.4.2 芯片详情模块 23
4.4.3 集群管理及详情模块 25
4.5 司南状态页面 27
4.6 任务管理 30
4.6.1 任务提交页面 30
4.6.2 我的任务页面 33
4.6.3 任务详情页面 34
4.6.4 实时任务状态页面 35
4.6.5 用户任务管理页面 36
4.6.6 任务报表页面 37

5 API 参考 **39**
5.1 PilotOS 登录 . . . 39
5.2 提交量子芯片计算任务 . . . 40
5.3 查询任务 . . . 43
5.4 获取队列信息 . . . 44
5.5 获取芯片信息 . . . 45
5.6 提交集群模拟计算任务 . . . 46
5.6.1 Configuration 信息说明及请求示例 . . . 47
5.7 集群模拟计算任务状态查询 . . . 49
5.8 获取指定后端的任务排队信息 . . . 50
5.9 获取芯片校准时间 . . . 50
5.10 暂停芯片调度服务 . . . 51
5.11 获取芯片调度服务状态 . . . 51
5.12 手动进行 M3 采样 . . . 52
5.13 Session 机制任务管理 . . . 52
5.13.1 请求会话 . . . 52
5.13.2 在会话中提交任务 . . . 53
5.13.3 关闭会话 . . . 53
5.14 获取芯片最优量子比特块 . . . 54
5.15 获取任务的优先级 . . . 55

6 消息推送 **56**
6.1 芯片校准开始信息 . . . 56
6.2 芯片校准结束信息 . . . 56
6.3 芯片新配置信息 . . . 57
6.4 芯片人工维护开始信息 . . . 57
6.5 芯片人工维护结束信息 . . . 58

7 类型定义与错误码 **59**
7.1 任务类型 . . . 59
7.2 计算后端类型 . . . 59
7.3 任务状态类型 . . . 60
7.4 错误码 (ErrorCode) . . . 60

8 运维与管理 **62**
8.1 系统维护 . . . 62
8.2 故障排除 . . . 62

1 简介

1.1 手册目的

本用户手册旨在为量子操作系统“本源司南”（Origin PilotOS）提供全面的操作指南和技术细节。主要受众包括以下群体：

- **量子计算研究团队：**专注于量子计算理论、算法优化以及量子软硬件集成的研究人员。他们使用司南作为实验平台，测试和验证新型量子算法或架构。
- **量子信息科学家：**量子密码学、量子通信和量子网络等领域的研究人员，利用司南来管理和编排复杂的实验任务。
- **量子应用开发者：**开发量子程序或工具的个人和团队，利用司南进行高效的编程、调试和算法测试。
- **科技公司和初创企业：**专注于量子硬件开发或量子软件平台的企业，使用司南管理硬件资源、优化性能，或将其作为提供量子计算服务的基础设施。
- **国家实验室和政策制定者：**政府资助的研究实验室，使用司南推进量子技术研究并支持国家创新战略。

本手册旨在帮助这些用户有效地理解、操作和管理司南系统。

1.2 系统概述

量子计算是一种结合了量子力学与计算机科学的新型计算范式。它利用量子比特（qubit）作为计算的基本单位，利用叠加、纠缠和测量等量子现象，在特定问题上实现超越经典计算的指数级加速。量子计算在非对称加密破解、量子化学、金融建模和机器学习等领域具有显著优势。

近年来，量子芯片的规模和质量取得了快速发展。最新的超导量子芯片已达到 443 量子比特（例如 IBM 的”ibm-seattle” 芯片），预计未来几年超导芯片将突破 1,000 量子比特。同时，基于不同物理平台的量子芯片技术也在并行发展，各自在计算效率、保真度和稳定性方面具有独特的优势。

在大规模量子比特下管理量子计算资源、高效调度任务，以及实现异构多后端量子计算和量子-经典混合计算，是实用化量子计算系统发展中必须解决的关键挑战。司南是中国首个量子计算操作系统。其核心目标是实现高效的量子-经典混合计算。系统由量子线路编译与优化、量子资源管理、量子任务调度、系统监控和后端量子计算服务等关键组件构成。它支持跨异构量子后端的集成计算，并通过开源量子编程框架 **PyQPanda** 为用户提供全栈式量子计算服务。

1.3 术语与缩写

以下是本手册中使用的关键技术术语和缩写：

- **QPU (Quantum Processing Unit，量子处理单元)：**相当于经典 CPU 的量子对应物，负责在量子硬件上执行量子操作。
- **NISQ (Noisy Intermediate-Scale Quantum，含噪声中等规模量子)：**指当前的量子计算时代，其特征是设备拥有数十到数百个量子比特，且容易受到噪声和错误的影响，限制了其计算可靠性和可扩展性。
- **Qubit (Quantum Bit，量子比特)：**量子信息的基本单位，能够以 |0> 和 |1> 态的叠加形式存在。
- **Quantum Superposition (量子叠加)：**允许量子系统同时处于多种状态直到被测量的原理。
- **Quantum Entanglement (量子纠缠)：**一种现象，即两个或多个量子比特之间形成关联，即使相隔很远，其中一个的状态也无法独立于其他比特进行描述。
- **PyQPanda：**一个开源的量子编程框架，允许用户在各种量子后端编写、模拟和执行量子程序。

2 安装与部署

本章详细介绍司南系统的自动化部署流程。自动化部署主要基于自动化脚本“AutoDeploy.py”，该脚本封装了镜像管理和数据库初始化步骤，用户只需要执行该脚本，即可完成整个安装操作。

2.1 硬件要求

硬件主要涉及三类组件：CPU、DRAM 和存储。对于 CPU，我们建议使用 AMD Zen 5 系列或 Intel Arrow Lake/Diamond Rapids 系列处理器，或更新的型号。为了充分发挥 PilotOS 的性能潜力，建议使用至少 16 核的处理器。鉴于强大的 CPU 能力，应搭配至少 64 GB 的 DRAM（DDR5，>= 5600 MHz）。此外，PilotOS 在安装和运行期间需要数百 GB 的磁盘空间；因此，我们建议为 PilotOS 预留至少 1 TB 的存储容量。关于网络接口卡（NIC），所需的带宽取决于工作负载的规模。由于量子态空间的指数级增长，我们建议 NIC 带宽至少为 10 Gbps，以避免网络瓶颈影响整体计算效率。

2.2 软件依赖

在开始部署之前，请确保宿主机满足以下软件环境要求：

- **操作系统**：建议 Ubuntu Server 22.04 LTS
- **Python 环境**：Python $\geq$ 3.8.0
- **容器引擎**：Docker $\geq$ 20.10.0（需安装 docker-compose 组件）
- **MySQL 数据库服务**：8.0 版本以上
- **MongoDB 数据库服务器**：6.0 版本以上

2.3 部署准备

司南镜像部署包解压后的文件结构如下所示：

Listing 1: 部署包文件结构

```
./package
  AutoDeploy.py # 自动化部署脚本
  build-dep.sh
  docker-compose.yml
  mysqlInit # 数据库初始化脚本目录
  pilotos:x.x.iso # x.x版本的司南镜像文件
  PilotOS-Config # 配置文件目录
  requirements.txt # Python依赖表
  Upgrade.py
  Util.py
```

2.3.1 解压部署包

执行以下命令解压部署包（请根据实际文件名修改）：

```
tar -zxf pilotos-x.x.tar.gz
```

2.3.2 安装部署依赖

使用 pip 安装部署脚本所需的 Python 依赖库：

```
sudo pip3 install -r requirements.txt
```

注意：必须使用 `sudo pip` 进行安装，以确保依赖包安装在系统环境中。后续执行自动化脚本时也必须使用 `sudo python`，否则会报找不到依赖包的错误。

2.4 部署流程

2.4.1 数据库初始化

在执行初始化前，请先修改 `mysqlInit` 文件夹下 `config.yaml` 中的数据库配置，将其改为实际环境中的数据库信息。修改完成后，在解压根目录下执行：

```
sudo python3 AutoDeploy.py initDB
```

2.4.2 修改配置文件

根据实际部署环境，修改以下配置文件中的 MySQL 和 MongoDB 配置：

```
sudo vim ./PilotOS-Config/config.json
```

2.4.3 加载镜像文件

执行以下命令加载司南 Docker 镜像：

```
sudo python3 AutoDeploy.py loadImage
```

2.5 服务启动与验证

2.5.1 启动服务

执行以下命令启动司南镜像服务和后台升级服务：

```
sudo python3 AutoDeploy.py start
```

启动注意事项：

- 每次启动前，脚本会自动检查服务是否已运行。如果已存在，则不会重复启动。
- **强制重启说明**：如果在特殊情况下需要强制重启升级服务，**必须**使用 `sudo kill` 杀死进程，**严禁**使用 `kill -9`。
- 使用 `kill -9` 会导致 `/var/run/autodeploy_upgrade.pid` 文件未被删除，从而导致下次启动失败。只有当该 PID 文件不存在时，升级服务才能正常启动。

2.5.2 验证访问

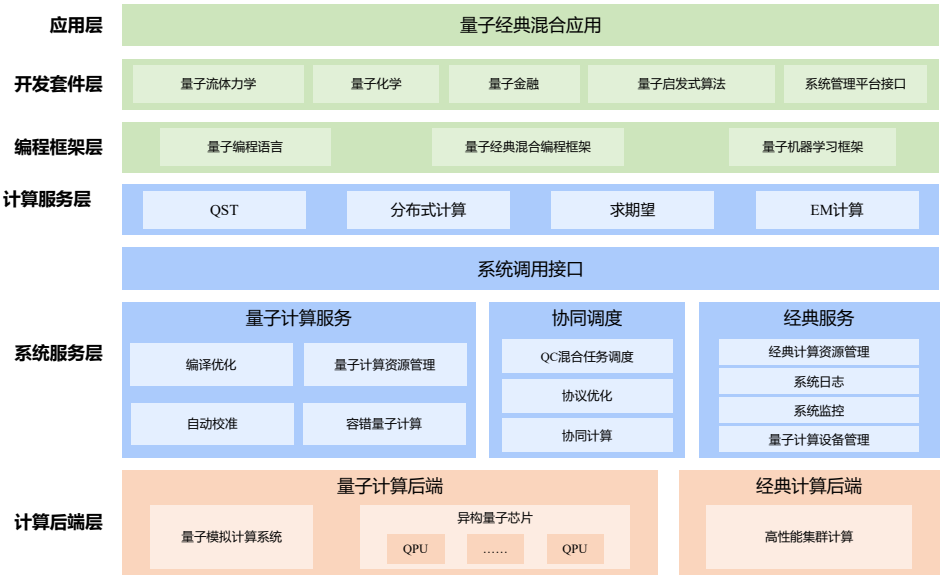
服务启动成功后，可以通过浏览器访问管理端进行测试：

- **访问地址**：`http://< 服务器 IP>:10020`

3 系统架构

3.1 整体架构图

本源司南的整体架构如下图所示。



应用层：主要指用户编程空间，用户可以在本源司南系统之上开发各种应用程序。

应用开发工具包：本源司南为上层用户提供了多种开发工具包，这些工具包已集成到 Pilot 系统中。部署司南系统后，用户可以直接基于这些工具包进行二次功能开发或快速验证量子算法。目前包含的工具套件有：ChemiQ（量子计算化学软件包）、QFinancial（量子金融开发工具包）、QCFD（量子计算流体动力学开发工具包）、Quantum Heuristic Algorithm Toolkit（量子启发式算法工具包）以及 System Manager Interface（系统管理 API）。

内置编程框架：本源司南系统提供三种集成的编程框架：

a. 量子编程框架。专用于量子程序开发的框架，提供：1) 高级量子编程语言；2) 用于执行和验证量子算法的模拟接口；高级量子线路工具，包括线路打印、等价性检查、线路反转以及转换为其他量子编程语言（如 Qiskit, Cirq）。

b. 量子-经典混合编译框架。通过以下方式优化混合量子-经典程序：1) 分析并调度量子任务与经典任务之间的执行顺序；2) 应用编译器级优化以提高整体算法效率和资源利用率。

c. 量子机器学习 (QML) 框架。支持量子与混合机器学习模型的无缝开发，具备：1) 基于 Python 的用户友好 API，支持自动微分和动态计算图；2) 在多种硬件后端（量子处理器、CPU、GPU）上的统一执行；3) 内置用于加速训练和推理的高性能数值库；4) 兼容主流机器学习框架（如 PyTorch, TensorFlow），便于集成到现有的 ML 工作

流中。

系统接口：上层应用访问本源司南系统的统一入口。它强制隔离系统层与应用层，确保用户空间应用只能有限地、受控地访问系统资源，从而维护系统的稳定性和安全性。

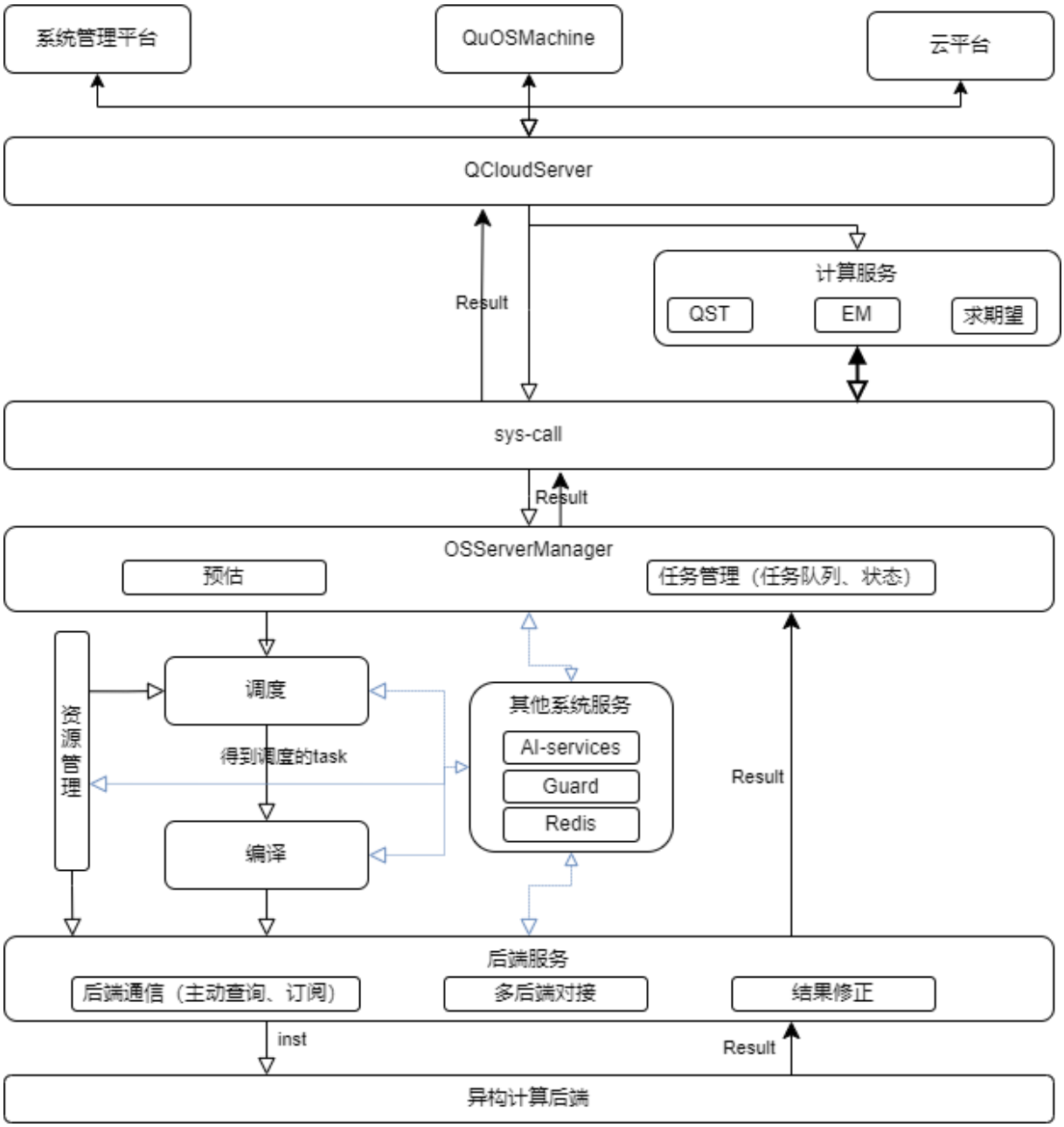
计算服务层：处理具有专用处理逻辑的特定业务场景，将应用级计算任务与核心系统计算解耦，以确保整体计算的稳定性。目前集成的服务包括：1) 量子态层析 (QST)，2) 期望值估计，3) 误差缓解 (EM) 计算，4) 分布式量子线路计算。

系统服务层：本源司南量子操作系统构建在一组核心、可互操作的模块之上：系统管理模块、调度服务模块、编译服务模块、资源管理模块和后端服务模块。这些模块通过安全的内部系统通道进行无缝通信，实现协调的工作流执行——从任务提交和编译到资源分配和物理硬件上的量子计算。该架构设计关注点分离且松耦合，确保了跨异构量子后端的高可靠性、可扩展性和兼容性。

计算后端：本源司南原生支持多种量子计算后端，包括超导量子处理器、量子模拟器以及经典计算后端。

3.2 执行流程

下图展示了 Pilot 系统中量子计算任务的端到端工作流。



任务提交：用户可以通过以下方式提交量子计算任务：

- **QPandaLite**：构建在 Pilot Web 管理界面之上的 Python SDK（通过 Nginx 服务）；
- **PilotOSMachine**：提供 Python 和 C++ 版本；
- **Origin Cloud Platform（本源云平台）**：供外部访问的公共 HTTP API。

所有任务均由 QCloudServer 层接收并进行持久化管理。

任务预处理与路由：对于专用任务（如量子态层析 (QST)、期望估计、EM 计算），QCloudServer 会将其转发给专用的计算服务进行预处理，然后再通过系统 API 调用 Pilot 内核。

Pilot 中的任务编排：

- **O**SServerManager 模块作为中央协调器，管理内部模块并维护任务队列。
- **每**个任务的状态通过共享的状态映射表进行跟踪。随着任务在各模块间的流转，其状态更新会在 OSServerManager 中汇总。
- **任**务完成（成功、失败或取消）后，所有状态数据将传输至 QCloudServer 进行持久化存储，并从 OSServerManager 中移除——确保 OSServerManager 仅保留活动任务的状态。
- **O**SServerManager 还提供资源预估功能，预测所需的量子比特数、线路深度和运行时间，以支持调度决策。

调度：经过预估的任务进入调度队列。调度器根据以下因素分配资源：

- **任**务资源/运行时间需求，
- **当**前后端的可用性。

支持的策略包括：

- **先**来先服务 (FCFS)，
- **最**高响应比优先 (HRRN)，

可通过 Pilot 的配置文件进行配置。

编译与执行：调度后的任务通过优化通道被编译为特定后端的指令。后端服务随后：

- **将**任务分发至目标量子硬件（支持多种架构），
- **获**取结果和后端状态，
- **应**用结果修正算法以减轻硬件噪声并提高准确性。

这种精简的架构确保了跨多样化量子计算工作负载的稳定性、可扩展性和高效的资源利用。

3.3 系统服务层核心模块

3.3.1 QCloudServer 模块

Pilot QCloudServer 模块作为用户访问 Pilot 量子操作系统的唯一官方入口。它暴露了安全的基于 HTTPS 的 API 接口，确保所有通信都是加密且可靠的。

- **统一访问网关**。所有用户请求——包括量子任务提交、状态查询和结果获取——必须通过 Pilot 云服务路由。不暴露其他直接访问系统的路径，以确保安全性和集中控制。
- **请求验证与安全强制**。收到请求后，服务会验证用户身份、API 权限和请求参数。格式错误、未授权或潜在的恶意请求将在到达核心系统组件之前被拒绝。
- **持久化任务管理**。验证通过的请求将持久存储在云服务的数据库中，包括任务元数据，如用户 ID、提交时间戳、任务类型和配置参数。这实现了全生命周期的跟踪、审计和恢复。

- **安全内部调度**。任务和查询通过安全的内部系统通道转发至下游 Pilot 模块（如 QCloudServer, OSServerManager）进行编译、调度、执行或状态查找。
- **结果交付与状态同步**。完成后，计算结果或查询响应通过相同的内部通道中继回云服务，然后再通过 HTTPS API 交付给用户——完成端到端工作流。

3.3.2 ComputerServer 模块

ComputerServer 模块位于 PilotOS 云服务层和系统内核层之间，作为一个中间件模块。其主要目的是将通用但不常用的计算功能从核心执行流程中解耦。这种设计不仅减轻了系统内核的负载，还增强了整体的可扩展性和功能敏捷性。目前，计算服务提供以下关键能力：

- **期望值评估**用于量子线路
- **全线路保真度估计**

该服务采用模块化架构，由四个核心组件组成：

- **任务预处理**。解析传入的任务请求，验证输入参数，并生成标准化的内部任务表示。
- **子任务展开**。根据语义要求，自动将高级任务分解为多个可并行或顺序执行的子任务。
- **子任务执行**。将子任务分发到底层量子或经典计算资源进行实际计算。
- **结果聚合**。通过归一化、加权或统计融合将子任务输出组合成最终面向用户的结果。

该框架设计用于易于扩展，能够在保持与主 Pilot 系统松耦合的同时，无缝集成新的特定领域计算功能。

3.3.3 OSServerManager 模块

OSServerManager 模块作为 Pilot 系统内的中央协调组件。它通过标准化的系统调用接口 (SysCall) 与上层应用通信，负责两个关键功能：任务资源预估和端到端任务队列管理。

- **任务资源预估:** 在量子任务排队执行之前，OSServerManager 对提交的量子线路进行静态分析以预估其资源需求。这包括：所需量子比特数、线路深度、量子门数量。基于此分析，系统确定当前是否有足够的资源来接受该任务。
- **任务队列管理:** 通过资源预估的任务被放入统一的调度队列。OSServerManager 持续跟踪每个任务在执行流程中的进度并实时更新其状态。
  这些状态更新会立即推送给您的应用程序，使您能够细粒度地了解任务执行的每个阶段。

- **任务数据的持久化存储:** 任务完成后，Pilot 自动使用分层存储策略持久化其数据：结构化元数据（如任务 ID、后端 ID、开始/结束时间戳）存储在 MySQL 中以进行高效查询和监控。大型非结构化数据（如完整量子线路、原始测量结果）保存到 MongoDB 中以支持灵活访问和扩展。这种设计确保了数据的可靠性和可审计性，同时保持核心系统的轻量级和响应速度。

3.3.4 ScheduleServer 模块

ScheduleServer 模块接收已通过资源预估的任务——连同其关联的资源需求——并将符合条件的任务分发给编译模块进行处理。默认情况下，系统采用最高响应比优先 (HRRN) 调度策略，该策略通过基于等待时间和预估执行时长来确定任务优先级，从而平衡公平性和吞吐量。

调度工作流：

- **任务入队:** 任务在 OSServerManager 中通过资源预估后，会被分配一个初始优先级。该值由 OSServerManager 根据可配置策略（如任务类型、用户等级或业务上下文）确定——它不是固定为零。然后任务被推入调度器的优先级队列，该队列实现为最大堆，以确保当前响应比最高的任务始终位于顶部。
- **动态优先级更新:** 任务的有效调度优先级由其响应比控制，随着任务在队列中等待时间的增加，其响应比会增加，从而随时间推移获得更高的优先级。这防止了长时间等待的任务被饿死。
- **调度决策逻辑:** 在每个调度周期中，系统执行以下步骤：
  1) 检查堆顶任务的资源需求是否能被当前可用资源满足。
  2) 如果资源充足：任务从队列中弹出并转发至编译模块。
  3) 如果资源不足：不分发任务；相反，更新所有排队任务的响应比（优先级）。
  4) 无论结果如何，在周期结束时刷新优先级队列以准备下一次调度事件。

3.3.5 QCompileServer 模块

本源司南提供了一个全面、灵活且高性能的量子线路编译与优化框架。该框架抽象了底层量子硬件的复杂性，使量子软件开发者能够专注于算法设计和应用创新——无需了解不同量子处理器架构的具体细节。

主要优势：

- **硬件无关开发:** 开发者只需提交标准量子线路（例如 QASM 或内部中间表示）。编译器自动将线路适配到目标后端，实现跨支持量子硬件平台的 “一次编写，到处运行”。
- **异构多后端支持:** 当前版本支持两种主流量子计算模态：超导和离子阱处理器。根据用户指定的目标后端，系统动态选择合适的编译流程以生成完全兼容的原生指令集。

- **并行编译加速:** 基于并行编译框架构建，服务可以并发处理多个任务或将大型线路分解为片段进行并行优化——显著提高整体编译吞吐量，特别是对于批量作业或高复杂度线路。

编译与优化能力：Pilot 编译器集成了一套先进的优化算法，每个算法都通过标准化、模块化接口封装，以促进重用、组合和未来的扩展。核心优化技术包括：

- **多控门分解**
- **门集转换**
- **量子线路优化**
- **量子比特映射与拓扑路由**

3.3.6 ResourceManager 模块

资源管理模块负责接收来自量子计算后端的资源更新通知或来自上层系统组件的资源配置消息。它维护可用资源的最新清单，并将此信息与系统中的其他服务同步。

核心职责：

- **处理资源更新和配置请求:** 接收有关资源变更（如后端可用性、量子比特状态）或显式资源配置命令的消息，并相应地更新内部资源注册表。
- **服务资源信息查询:** 响应其他系统组件的请求，按需提供当前且准确的资源数据。

3.3.7 BackendServer 模块

BackendServer 模块负责管理多个异构量子计算后端单元（如超导、离子阱），并处理计算任务的分发和执行结果的接收。作为 Pilot 系统与底层量子控制服务之间的中央通信枢纽，该模块采用混合消息模型来平衡可靠性、实时响应性和可扩展性。

通信架构：为了满足多样化的操作需求，后端服务模块在与量子控制服务交互时集成了两种 ZeroMQ 通信模式：

a. Router-Dealer 模式（请求-应答模式）

- **特征:** 面向连接、双向、同步请求-响应。
- **主要用例:** 将量子计算任务分发至特定后端；接收任务执行结果；查询实时任务状态；获取随机基准测试 (RB) 实验数据；交换心跳信号以进行链路健康监控和故障检测。
- **优势:** 确保有序、可靠的消息传递——是关键控制命令和结果获取的理想选择。

b. Pub-Sub 模式（发布-订阅模式）

- **特征:** 无连接、单向、异步、事件驱动。
- **角色:** Pilot 后端服务作为订阅者，持续监听来自控制服务的广播消息。

- **推送事件:** 典型的推送事件主要包括：主动任务状态更新（如异常终止、资源抢占）；芯片参数的动态变化（如 T1/T2 相干时间、门保真度）；自动校准例程完成时的通知；芯片资源可用性的实时更新（如量子比特故障、冷却恢复）。
- **优势:** 消除轮询开销并实现低延迟事件传播，增强系统响应能力。

4 可视化管理控制平台

本章节提供 PilotOS 可视化管理控制台的概览，包括界面结构和核心功能模块。可视化管理控制台提供覆盖用户账号管理、系统监控、资源调度以及完整任务生命周期操作的一体化可视化平台，使用户和管理员能够高效操作量子计算和集群资源。

主要功能包括：

用户账号与访问管理 系统提供完整的用户账号生命周期，包括：

- **用户注册**：通过输入个人信息和邮箱验证码创建新账号。
- **用户登录**：使用注册邮箱和密码登录。
- **用户管理（仅管理员）**：管理员可以查看用户列表、筛选用户、调整权限（Admin、VIP、Python API Access）、重置密码以及删除账号。

系统仪表盘 登录后，用户将进入仪表盘（Dashboard），可查看当前系统状态概览，包括：

- **任务概览**：展示不同状态的任务数量，支持选择时间范围（Today / This Month / This Year）。
- **系统资源使用情况**：以仪表形式显示 PilotOS 系统服务和 PilotOS 主机的 CPU 与内存使用情况。
- **芯片任务耗时分布**：展示任务在不同时间区间的运行时间分布。
- **文档访问**：快速链接到 PilotOS 技术文档。
- **快捷操作**：用于创建新任务和监控实时任务的快捷入口。

资源管理 资源管理页面提供量子芯片与集群的统一视图和控制界面：

- **芯片管理**：卡片式界面显示芯片名称、量子比特数量、计算模式和调度状态。用户可进入芯片详情，包括拓扑结构、校准参数、操作历史、芯片属性以及高级配置。
- **集群管理**：显示集群节点、资源使用情况、任务队列和配置摘要。用户可以打开详情视图查看节点状态和当前运行的任务。

Pilot 系统状态监控 Pilot 状态页面提供 PilotOS 的系统级监控，包括：

- CPU、内存、磁盘和数据库的资源使用卡片。
- 各模块的实时状态，包括每个模块的 CPU 和内存使用情况。
- 当前正在运行的任务列表。
- 管理员可执行的系统级操作，例如重启或关闭 PilotOS 系统服务。

任务管理　任务管理模块涵盖量子计算任务的提交、监控、历史查询以及报告导出：

- **任务提交**：支持多个模拟量子计算任务和真实量子芯片任务，包括参数配置、QProg 上传/编辑、结果查看，以及通过 Task ID 恢复任务。
- **我的任务**：显示当前用户提交的所有任务，支持筛选并进入任务详情。
- **任务详情**：显示执行时间戳、分阶段处理流程以及可视化结果。
- **实时任务状态**：监控系统中所有排队和运行中的任务，支持搜索和自动刷新。
- **用户任务管理**：用户可以取消任务；管理员可管理所有用户的任务。
- **任务报告**：支持根据时间范围、账号名、计算类型和状态筛选任务，可自定义字段并导出 CSV。

4.1　许可证验证

在首次使用 PilotOS 系统前，用户需完成许可证验证。许可证用于授权系统的整体使用权限，是进入平台的前置条件。

当系统尚未录入许可证，或当前许可证已过期时，将自动进入许可证输入页面，如图 1 所示，用户需在该页面填写有效许可证后方可继续使用系统。

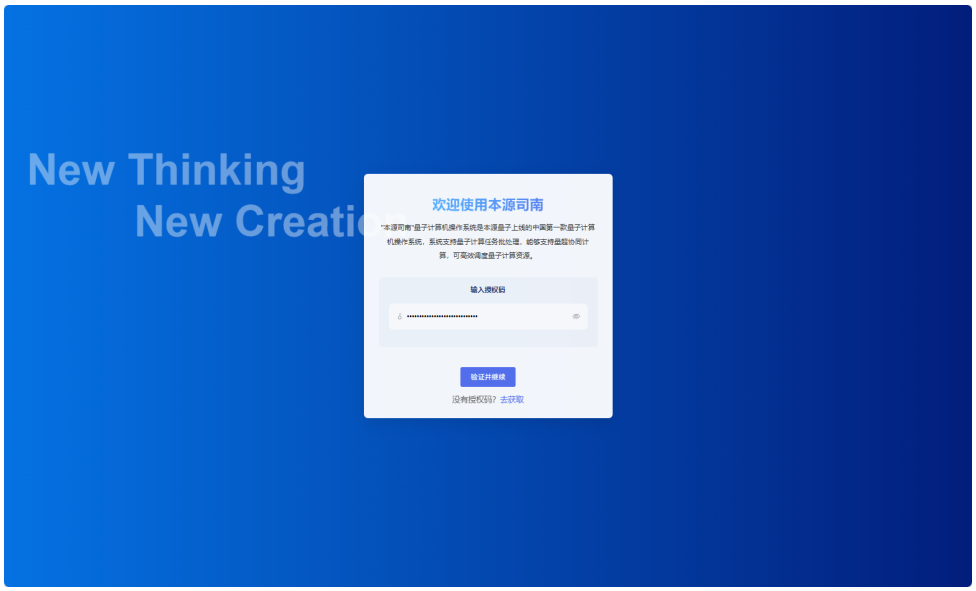


Figure 1: 许可证输入页面

使用流程　首次访问系统或许可证失效时，用户需按以下步骤完成验证：

1. 在许可证页面输入有效的许可证码；
2. 点击确认按钮提交；
3. 验证成功后进入系统登录页面；
4. 登录后即可正常使用系统功能。

若输入的许可证无效或已过期，系统会给出提示，用户需重新填写有效许可证。

许可证状态说明 从用户角度，许可证主要存在以下几种使用场景：

- **未录入许可证：**系统首次使用时需要录入许可证，否则无法进入系统。
- **许可证有效：**系统功能正常开放，用户可正常使用平台。
- **许可证即将到期：**系统仍可使用，但会在界面中提示剩余有效时间，建议提前更新许可证。
- **许可证已过期：**系统将暂停所有功能，仅允许重新录入许可证。完成更新后方可继续使用。

系统主界面顶部会显示许可证剩余有效天数，帮助用户及时了解当前授权状态。

注意事项

- 建议在许可证到期前提前更新，以避免系统服务中断；
- 许可证失效后，必须重新录入有效许可证才能恢复系统功能；
- 如无法完成许可证验证，请联系系统管理员或运维人员。

4.2 用户账号管理

4.2.1 用户注册

功能说明：此功能允许用户为 PilotOS 可视化管理控制台注册新账号。

注册流程：为完成注册，用户必须准确提供以下信息。请注意，每个邮箱地址只能关联一个账号——系统不允许使用相同邮箱重复注册。

表单字段：

- **名字**–输入您的名。
- **姓氏**–输入您的姓。
- **邮箱**–提供有效且唯一的邮箱地址。该邮箱将作为您的登录标识。
- **邮箱验证码**–请求并输入发送至您邮箱的一次性验证码。
- **手机号**–输入您的联系电话（可选，但推荐填写）。
- **护照号**–如适用，请填写您的护照号码。
- **身份证号**–如适用，请填写您的身份证号码。
- **所属机构**–填写您的单位、公司或机构。
- **密码**–创建符合系统要求的安全密码。

完成所有必填字段后，点击 **注册**按钮提交注册请求。系统将验证输入内容，并在所有条件满足的情况下为您创建账号。

注意：邮箱地址必须在系统中保持唯一。若尝试使用已注册邮箱进行注册，将导致错误提示。

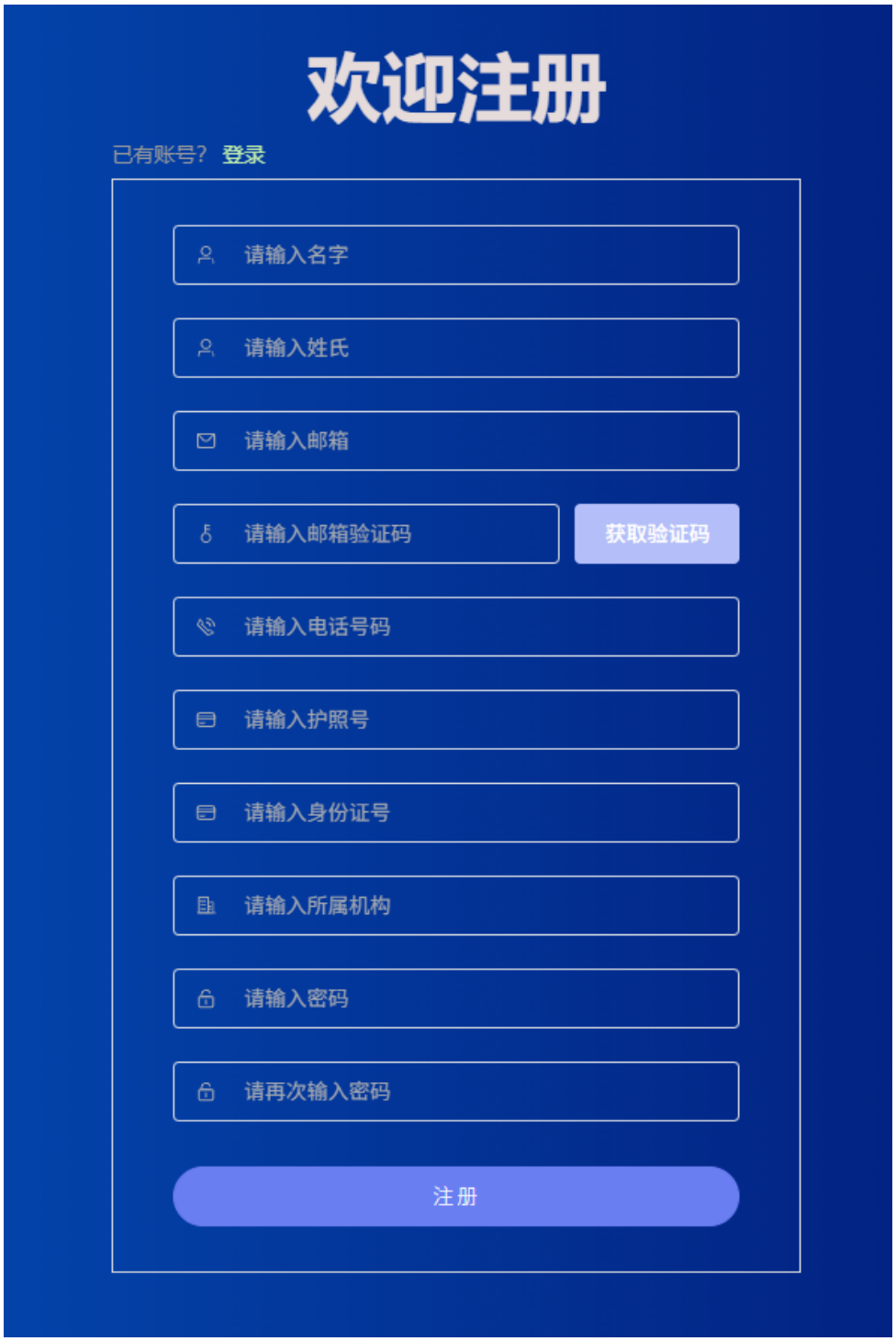


Figure 2: 注册表单界面

4.2.2 用户登录

功能说明：此功能允许用户使用已注册的账号凭证登录 PilotOS 可视化管理控制台。

登录流程：要访问系统，用户必须输入有效的登录信息并提交表单。验证成功后，系统会将用户重定向至主仪表盘。若凭证错误，将显示错误提示信息。

必填表单字段：

- **邮箱**–输入与已注册账号关联的邮箱地址。
- **密码**–输入您的账号密码。



Figure 3: 登录表单界面

成功登录后，系统将显示仪表盘界面。

4.2.3 用户管理

功能说明：此功能允许管理员账号管理用户信息，并控制用户在 PilotOS 可视化管理控制台中的访问权限。管理员可以修改账号状态、调整权限，并执行必要的账户维护操作。

操作流程：登录后，管理员可从顶部导航栏进入 **用户管理**页面。用户管理页面允许管理员根据特定条件筛选和显示用户信息。筛选项包括按照 **邮箱**、**Python 编程接口使用权限**以及 **用户角色**（普通用户或管理员）进行搜索。

可用操作：

- **权限调整**–使用 **VIP**、**设为管理员**和 **Python 编程接口使用权限**列下的滑块启用或禁用对应用户权限。调整这些滑块可控制用户是否具有 VIP 权限、管理员权限或 Python API 访问权限。

- **重置密码**–点击 **重置密码**按钮可将该用户的密码重置为系统默认值 123456。
- **删除账号**–点击 **删除**按钮可将所选用户账号从系统中永久删除。

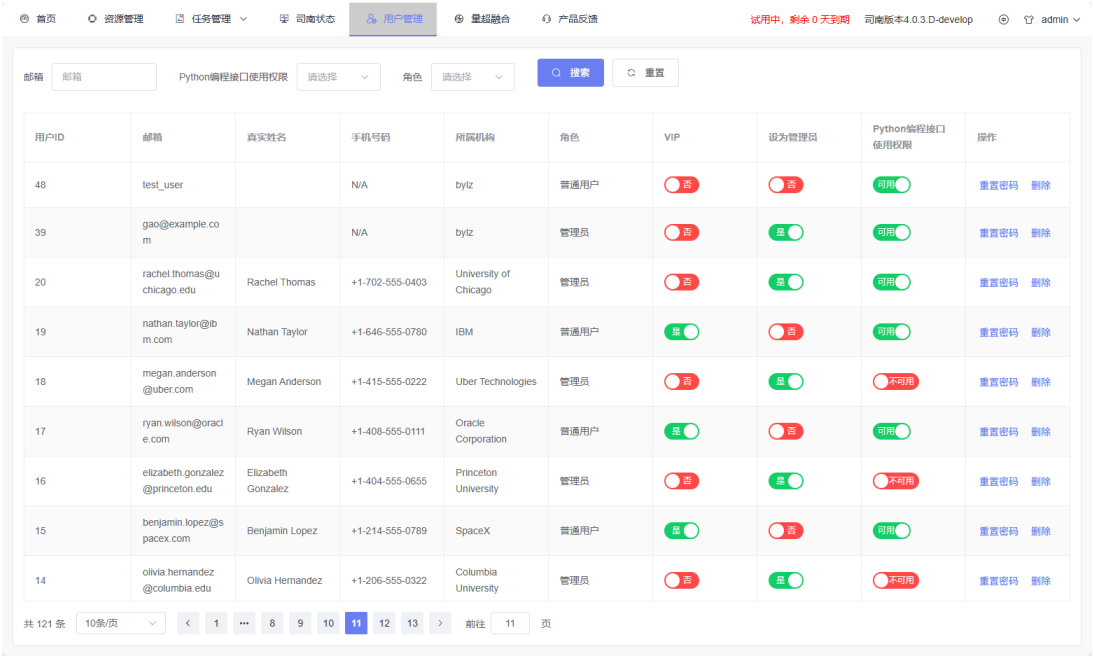


Figure 4: 用户管理界面

管理员可以通过调整表格中的滑块动态管理用户角色与权限。例如，启用**设为管理员**将授予管理员权限，而切换 **VIP** 或 **Python 编程接口使用权限**则控制对应功能使用。

注意：只有管理员账号可以访问**用户管理**页面。普通用户无权查看或修改其他用户信息。

4.3 系统仪表盘概览

页面概述：仪表盘页面作为 PilotOS 可视化管理控制台的综合概览页面，用于显示当前任务概览、资源使用情况及芯片任务耗时分布，同时提供快速访问文档和常用操作的入口。

登录成功后，用户将被重定向到仪表盘（见图 5），可即时掌握系统整体状态及任务分布情况。

4.3.1 任务概览

功能说明：此模块展示选定时间范围内所有任务的汇总统计，包括：

- 总任务数
- 待处理任务数
- 正在运行任务数
- 失败任务数

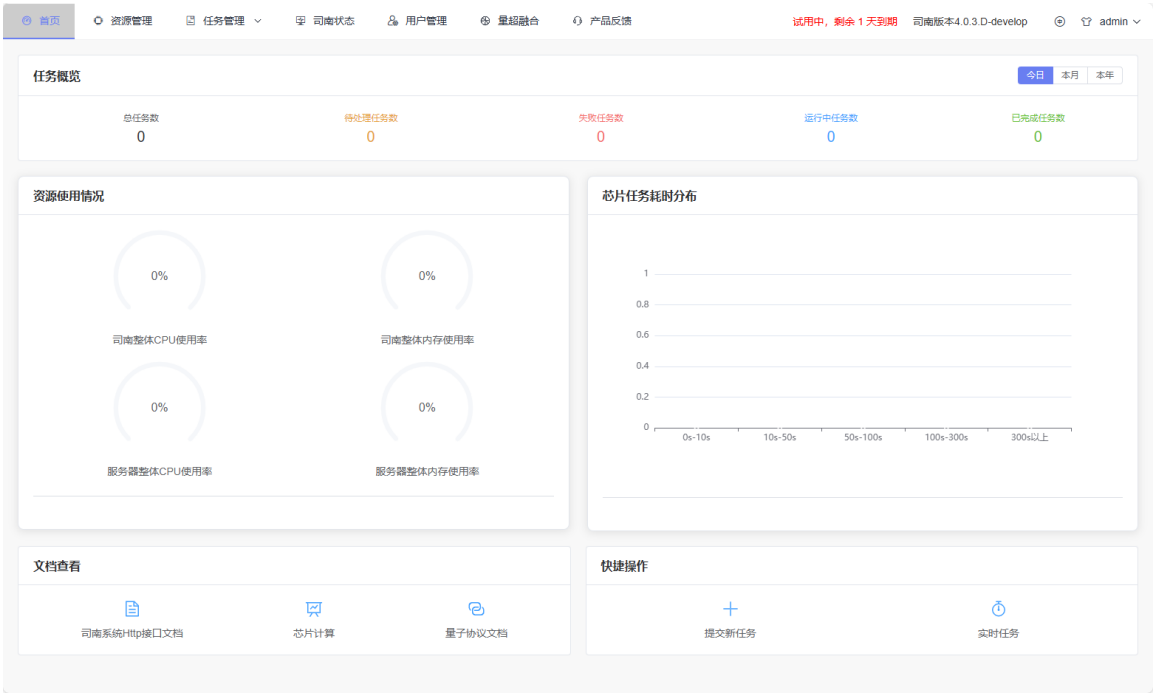


Figure 5: 系统仪表盘主界面。

- 完成任务数

用户可通过顶部时间选择器切换统计周期（今日 / 本月 / 本年），更改后数据自动刷新。

4.3.2 资源使用情况

该区域通过圆形仪表实时展示以下指标：

- PilotOS 系统服务 CPU 使用率
- PilotOS 系统服务内存使用率
- PilotOS 主机 CPU 使用率
- PilotOS 主机内存使用率

所有仪表均定期刷新，用于反映系统实时运行状态。

4.3.3 芯片任务耗时分布

功能说明：该模块以柱状图形式展示芯片工作负载在不同任务运行时间区间的分布情况。

横轴表示运行时间区间（如 0–10 秒、10–50 秒、50–100 秒），纵轴表示对应区间内的任务数量。

该图与任务概览模块共享时间选择器，切换统计周期后图表同步更新，用于分析系统整体执行效率。

4.3.4 文档访问

提供系统核心文档的快速访问入口：

- 司南系统 Http 接口文档
- 司南芯片计算操作手册
- 量子协议文档

点击图标将在新浏览器标签页中打开对应文档。

4.3.5 快捷操作

用于快速访问常用功能：

- **提交新任务**–跳转至任务提交页面。
- **实时任务**–跳转至实时任务状态页面。

这些快捷入口使用户能够直接从仪表盘高效完成日常操作。

4.4 资源管理页面概览

页面概述：资源管理页面是可视化管理控制台中用于统一监控与配置量子芯片及计算集群的核心界面。该页面集中展示所有计算资源的运行状态、基础配置及调度情况，并提供直观的交互式操作入口。

管理员可在此页面实时查看各类资源的在线状态与负载信息，同时支持通过图形化界面完成芯片资源的新增、编辑与删除操作，实现计算资源的动态接入与灵活管理。

页面包含两个主要标签页——**芯片管理**和 **集群管理**。进入资源管理页面后，默认显示 **芯片管理标签页**，重点展示量子芯片的状态与控制（见图 6）。

4.4.1 芯片管理

功能说明：芯片管理模块提供对当前注册在 PilotOS 系统中的所有量子芯片的全面概览。每张芯片卡片显示实时运行指标，并允许管理员直接在界面上控制调度操作。除系统自动注册的芯片资源外，该模块还支持用户手动添加自定义芯片资源。用户可通过“添加资源”入口录入芯片基础信息，实现新计算资源的快速接入；同时支持对已有芯片资源进行编辑与删除。如图 7 所示，每个芯片以独立卡片形式展示，显示芯片名称、配置及当前运行状态等关键信息，卡片右上角提供编辑与删除操作入口。

点击“添加资源”按钮后，将弹出芯片资源创建表单，如图 8 所示。用户根据页面提示填写芯片名称、芯片类型等必要信息，提交后即可完成资源注册。

操作方法：

- 点击芯片卡片打开详细信息抽屉，查看扩展配置及性能数据。
- 点击芯片卡片上的 **暂停运行**或 **恢复运行**按钮控制任务调度。

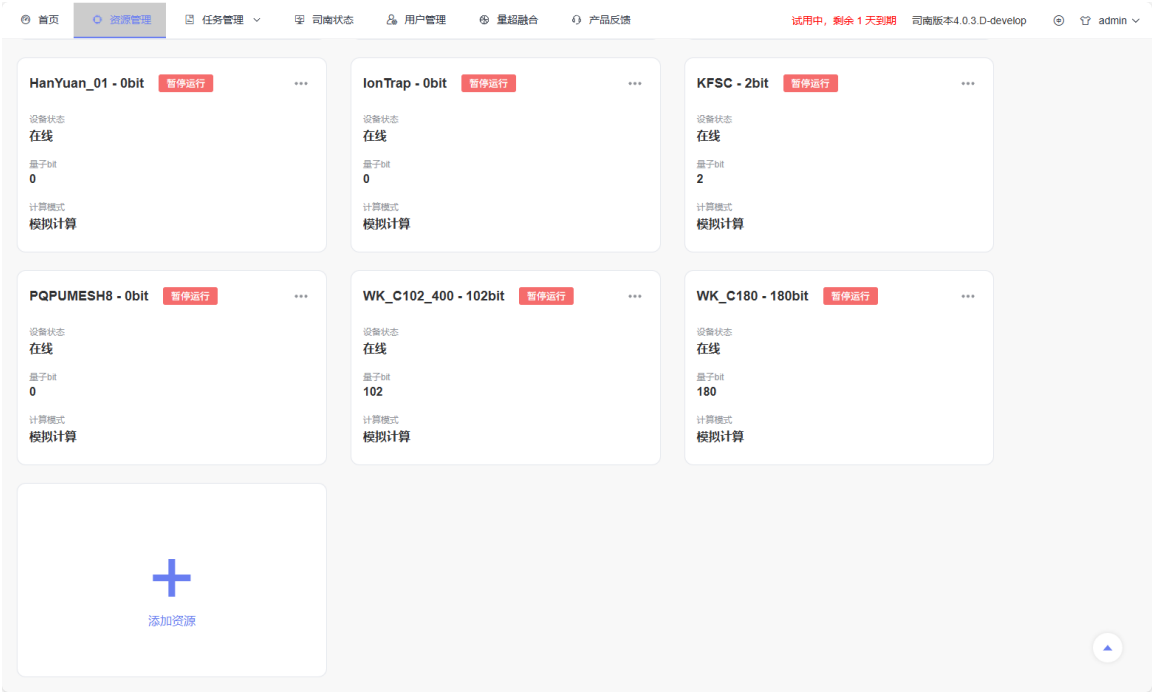


Figure 6: 资源管理页面主界面



Figure 7: 芯片管理标签页中的芯片卡片显示

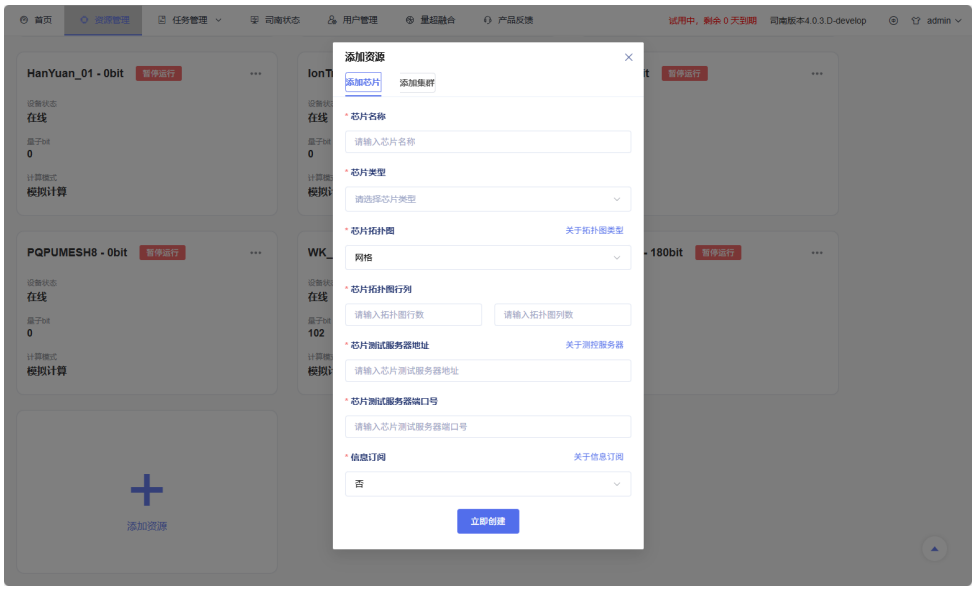


Figure 8: 添加芯片资源表单视图

- 点击页面顶部的 **添加资源**按钮，根据表单提示填写芯片信息并提交，完成新资源接入。
- 通过芯片卡片右上角的 **编辑**按钮修改已有资源配置。
- 通过芯片卡片右上角的 **删除**按钮移除不再使用的芯片资源。

4.4.2 芯片详情模块

功能说明：芯片详情模块提供全面界面，用于查看、分析和管理量子芯片配置及运行数据。用户点击芯片卡片后，右侧将滑出详情抽屉，包含两个功能标签页：**信息**与 **配置**，如图 9 所示。

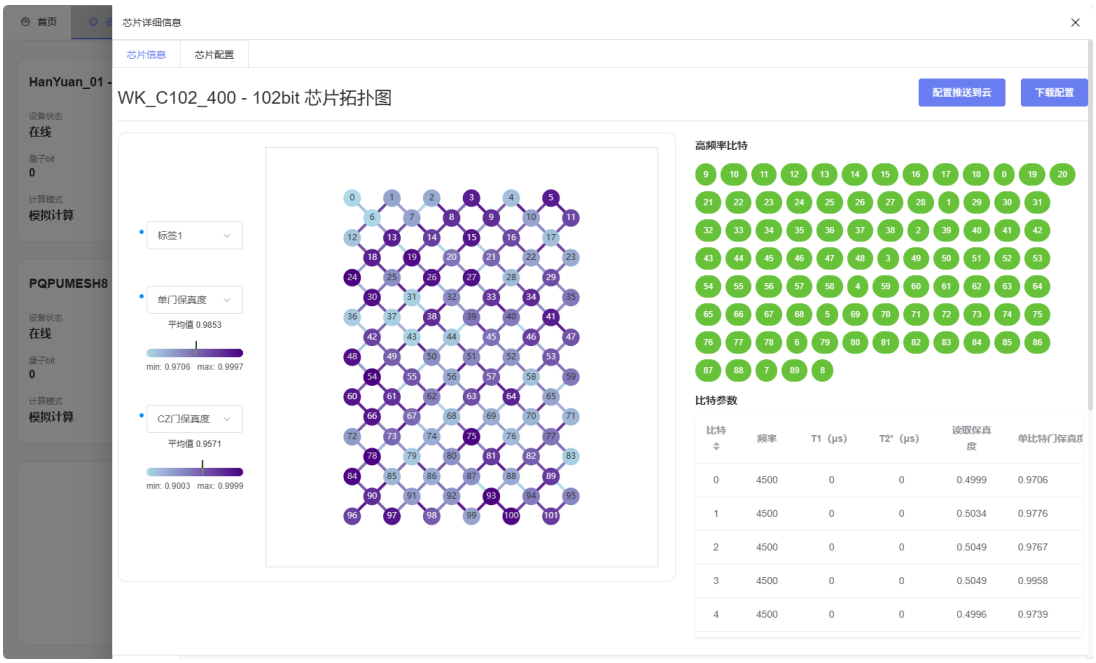


Figure 9: 芯片详情抽屉显示信息与配置标签页。

信息标签页 该标签页为量子芯片拓扑、量子比特参数、校准历史及系统配置的可视化主界面。

- **顶部区域：**显示芯片名称、量子比特数及拓扑类型，并提供：
  - **配置推送到云**–将配置数据手动同步至云端。
  - **下载配置**–导出当前芯片配置。
- **左侧面板–拓扑视图：**交互式拓扑图展示量子比特与连接关系。支持悬停或点击查看数据，点击量子比特可自动定位参数表对应条目。
- **右侧面板–量子比特参数表：**列出频率、T_1、T_2^*、读出保真度及单比特门保真度，高频量子比特自动标记。
- **底部标签页：**包含两个子标签，如图 10 所示：
  - **操作历史：**显示 M3 采样与校准日志，可手动触发采样。
  - **芯片属性：**展示逻辑门时序、噪声模型及系统元数据。

配置标签页 仅管理员可访问，用于高级配置与调度管理（见图 11）。**可配置选项：**

- 芯片事件通知设置（定时邮件提醒）
- 独占机时管理

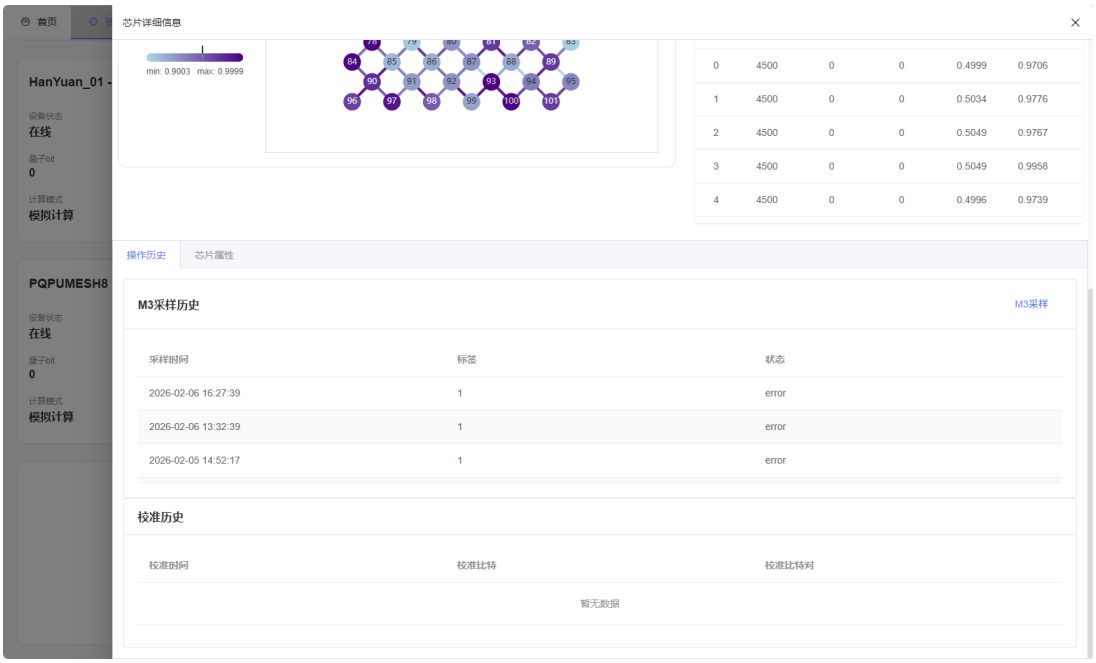


Figure 10: 底部标签页显示操作历史与芯片属性。

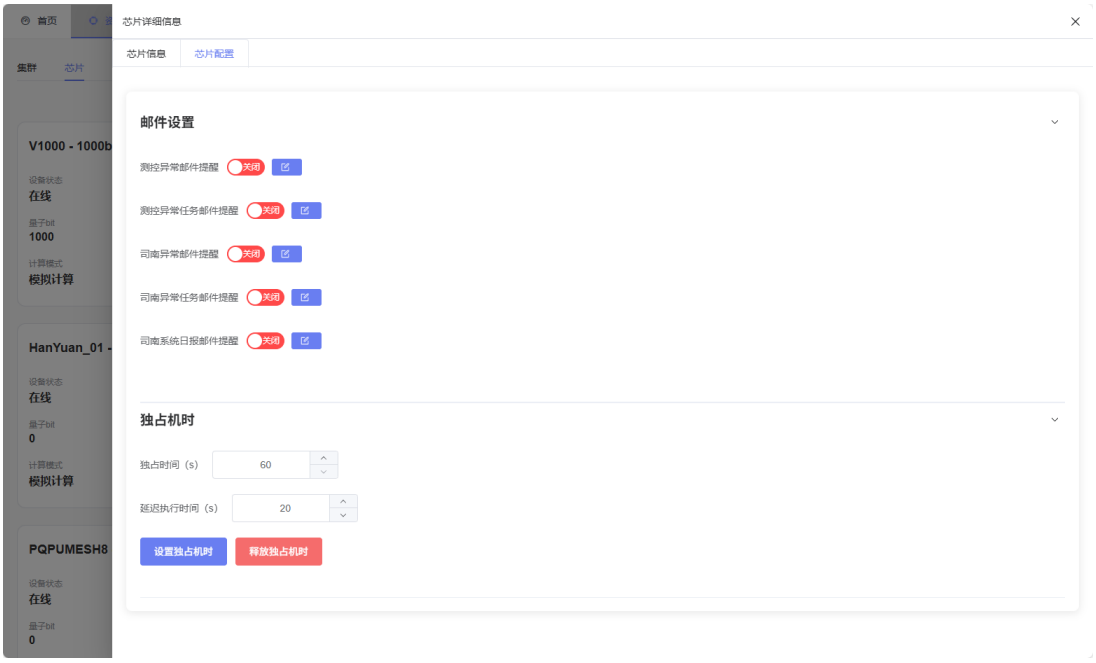


Figure 11: 高级芯片设置及调度管理配置标签页。

4.4.3 集群管理及详情模块

功能概述：集群管理及详情模块提供统一界面，用于监控和管理所有连接至 PilotOS 的计算集群。用户可在单一集成视图中查看集群概览、配置数据、节点信息及任务队列状态。

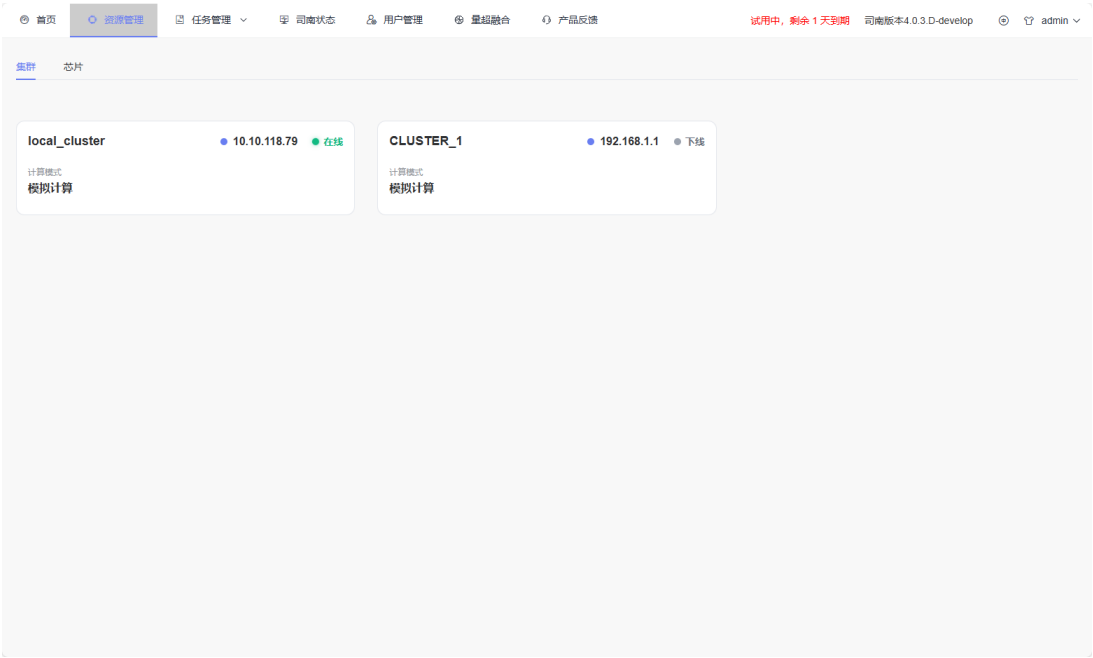


Figure 12: 集群管理概览界面

集群管理 如图 12 所示，每个集群以卡片形式展示其摘要信息，包括：

- **集群名称与状态：**顶部左侧显示集群名称，右侧显示主节点 IP 地址（如可用）。
- **调度状态：**在线 / 离线。
- **计算模式：**物理或模拟。

集群详情模块： 点击集群卡片后，右侧将滑出详情抽屉，提供实时集群性能及资源调度的深入视图。

如图 13 所示，该模块包含：

- **配置摘要：**显示关键硬件指标，包括节点总数、每节点内存、每节点 CPU 数量、最大子电路数量及主节点地址。
- **节点信息表：**列出各节点状态（运行或空闲）、CPU 负载、可用内存、最后运行时间及分区分配情况。
- **任务队列表：**提供实时任务数据，包括任务名称、状态、用户、运行时间、分配节点及所属分区。

操作指南：

- 通过节点信息表检查节点负载及内存使用情况。
- 监控任务队列以获取运行时间和调度信息。

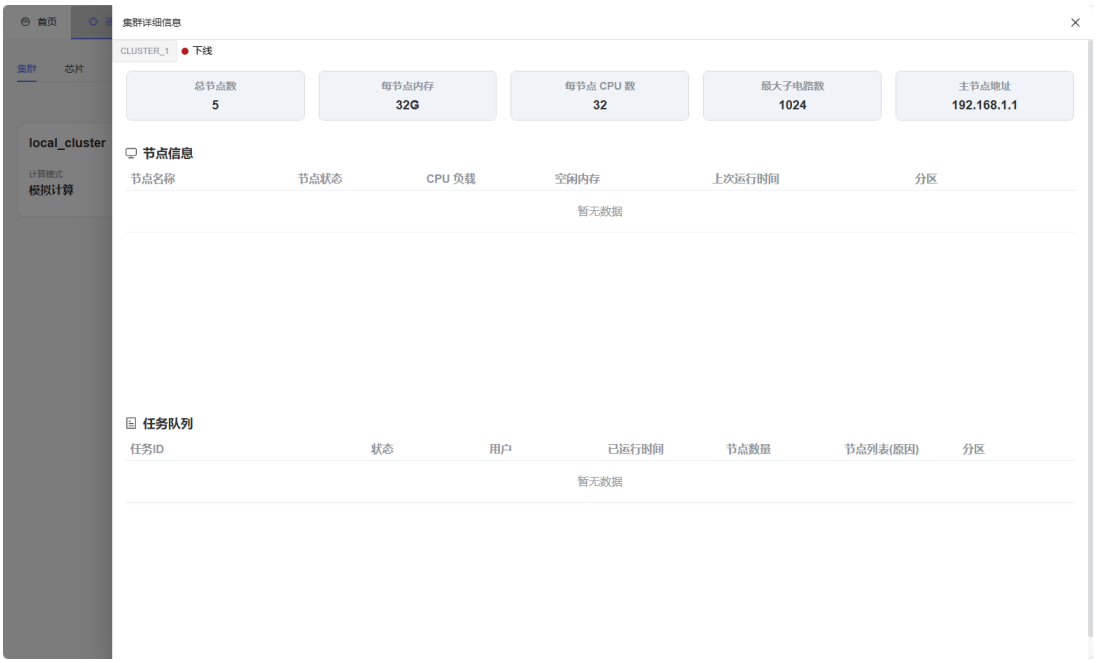


Figure 13: 集群详情模块

4.5 司南状态页面

功能概述：司南状态页面用于监控 PilotOS 系统服务的整体运行状态，包括系统资源使用情况、模块状态、任务列表以及系统控制操作。管理员可直接在该页面执行 **重启 Pilot** **关闭 Pilot** 或**升级司南**操作。

页面布局：如图 14 所示，司南状态页面包含以下功能模块：

- **系统状态概览模块：**页面顶部以卡片形式展示关键资源使用信息，包括：
  - 磁盘使用情况（已用空间 / 总空间）
  - PilotOS 系统服务 CPU 使用率
  - PilotOS 系统服务内存使用率
  - PilotOS 主机 CPU 使用率
  - 数据库空间使用情况

  当用户具有管理员权限时，该区域还提供 **重启司南**、**关闭司南**与**升级司南**选项。点击升级按钮可进入系统版本升级流程（详细说明参见本小节升级司南功能模块）。
- **模块状态表模块：**左侧卡片显示各 Pilot 系统模块的实时运行状态，包括：
  - 模块名称
  - 模块 CPU 使用率
  - 模块内存使用率
  - 服务状态（**正常** / **异常**）

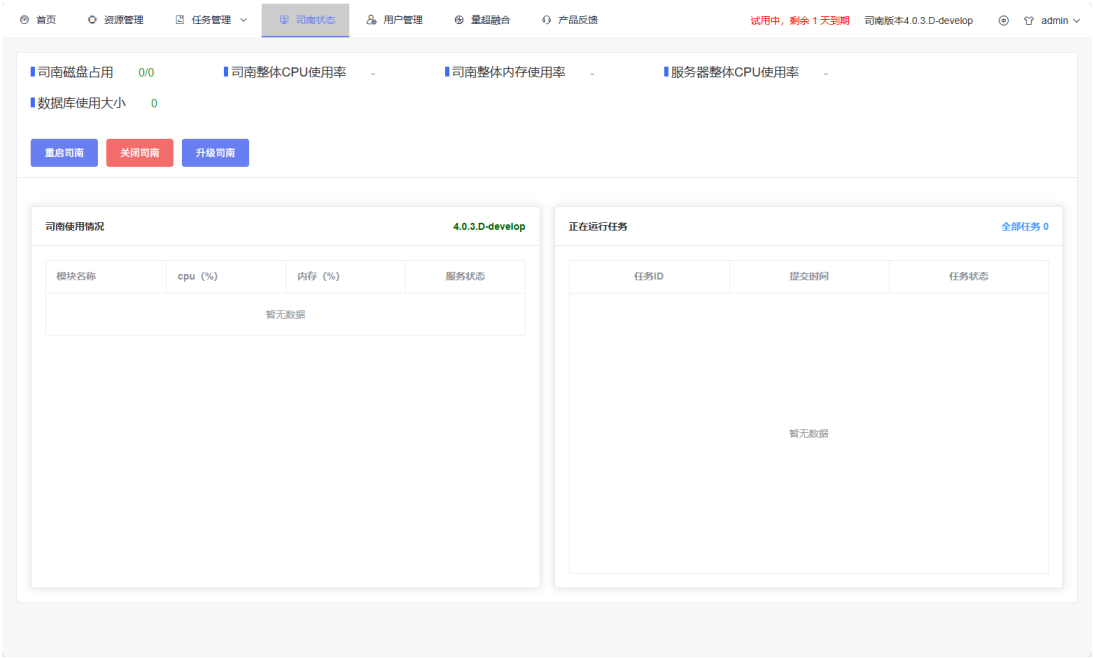


Figure 14: Pilot 系统状态页面概览

服务状态依据模块 CPU 使用率动态判定，并通过颜色标签进行直观展示。

- **任务状态表模块：**右侧表格展示当前运行任务的简要信息，包括：
  - 任务 ID
  - 提交时间
  - 任务状态

  卡片右上角显示任务总数。点击“全部任务”可跳转至实时任务状态页面查看详细进度。
- **升级司南功能模块：**升级司南功能用于系统版本更新。

  升级入口：用户可通过页面顶部的 **升级司南**按钮或顶部导航栏的司南版本按钮进入升级流程。该功能仅对具备管理员权限的用户开放。

  升级流程：升级流程分为以下步骤：

  1. 用户点击升级按钮，系统弹出升级方式选择对话框（如图 15 所示）：
     - **自动升级**：系统自动下载并完成升级（当前版本暂不提供该功能）。
     - **手动升级**：上传或放置升级包后执行升级（推荐）。
  2. 选择手动升级后，系统弹出升级引导对话框（如图 16 所示）。用户需将升级包（`.tar.gz` / `.zip`）放置到服务器指定目录：`/opt/PilotOS-Upgrade`。
  3. 点击 **已放置完成，开始校验**按钮，系统自动扫描指定目录下的升级包。
  4. 若存在多个升级包，系统弹出升级包选择对话框。用户可查看各升级包的版本号、构建信息、包大小和上传时间，并选择目标升级包；系统默认选中最新版本。

5. 系统自动校验所选升级包的完整性和安全性。校验通过后，显示新版本信息确认对话框。
6. 用户点击 **立即开始升级**按钮，系统进入升级状态。
7. 升级过程中，页面实时显示升级进度及当前操作状态（例如：通知升级服务、解压替换系统文件、重新启动司南服务等）。
8. 升级完成后，系统自动刷新页面并进入新版本。

注意事项：

- 升级过程中请勿关闭浏览器或刷新页面。
- 升级期间系统服务将暂时中断。
- 建议在系统空闲时执行升级操作。
- 请确保无重要任务运行后再开始升级。
- 若升级失败，请及时联系运维人员处理。
- 强制中断升级可能导致系统异常或无法启动。



Figure 15: 升级方式选择对话框

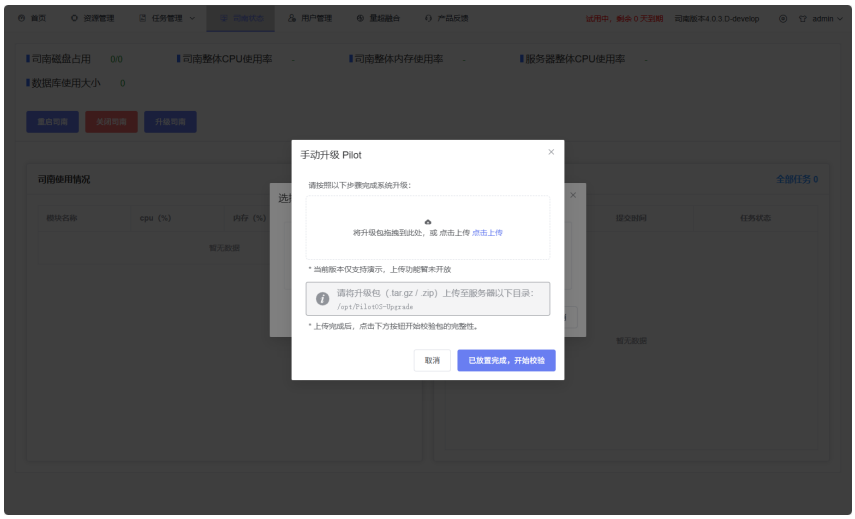


Figure 16: 升级引导对话框

4.6 任务管理

4.6.1 任务提交页面

功能概述：任务提交页面用于在 PilotOS 平台上配置、提交和查看量子计算任务，支持真实量子芯片与模拟器任务。用户可选择计算类型、选择后端、配置参数、上传或编辑量子程序（QProg），并查看执行结果。

页面同时支持通过任务 ID 查询历史任务。当找到匹配任务时，其参数、QProg 及结果（如可用）将自动恢复至表单中，实现任务复现或重新提交。

页面入口：

- 任务管理 → 任务提交
- 从 *我的任务*或 *任务详情*页面进入以恢复任务信息

页面结构：

如图 17 所示，任务提交页面包含四个主要功能区域：

- **任务查询与恢复区域（顶部）：**用于查询历史任务并恢复其参数。
  - **任务 ID 输入框：**输入已有任务 ID
  - **查询按钮：**查询任务并自动恢复参数、QProg 及结果（如可用）
- **任务配置区域（左侧）：**用于配置任务参数及选择计算后端。
  - **计算类型选择：**
    - 全振幅模拟
    - 噪声模拟
    - 部分振幅模拟
    - 单振幅模拟

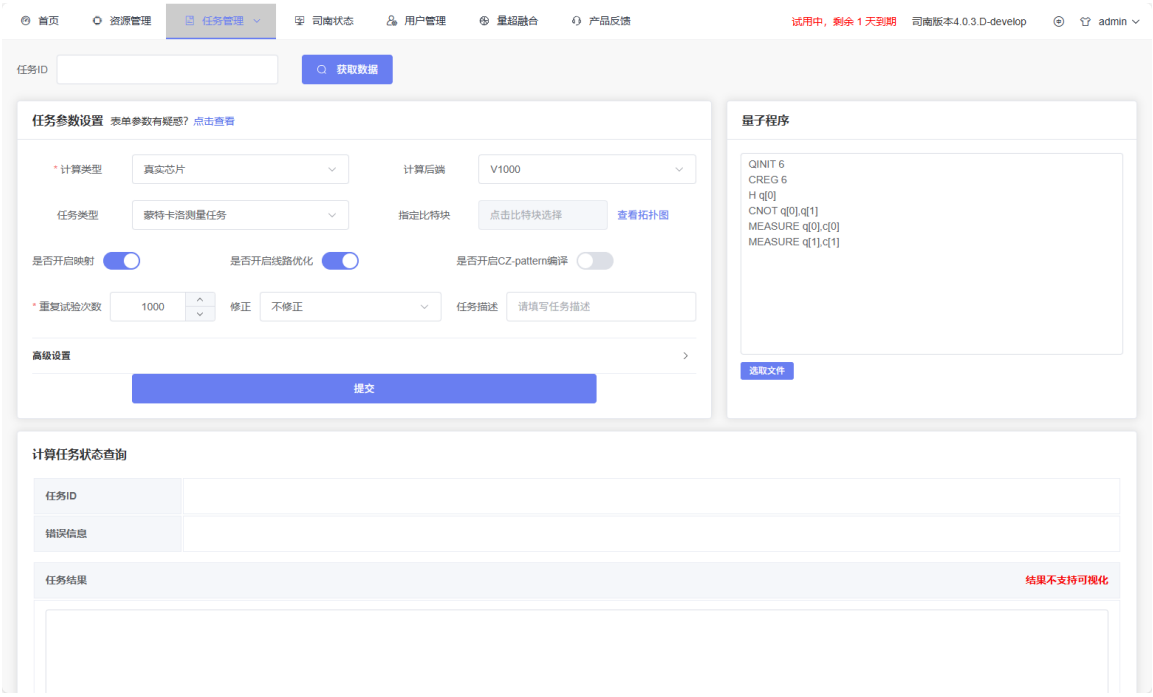


Figure 17: 任务提交页面概览。

  - MPS 模拟
  - 真实量子芯片
- **后端选择：**
  - 真实芯片任务：选择 **Chip ID**
  - 模拟器任务：选择 **Cluster ID**
- **通用参数：**
  - 测量类型
  - 采样次数
  - 目标量子比特
  - 指定量子比特块（通过拓扑对话框选择）
  - 映射选项
  - 噪声模型
- **芯片拓扑对话框（指定量子比特块）：**配置指定量子比特块时，可打开芯片拓扑对话框查看连接图，并选择或取消物理量子比特块，如图 18 所示。
- **高级设置（点击展开）：**
  - 工作点标签
  - 任务优先级
  - 回调地址

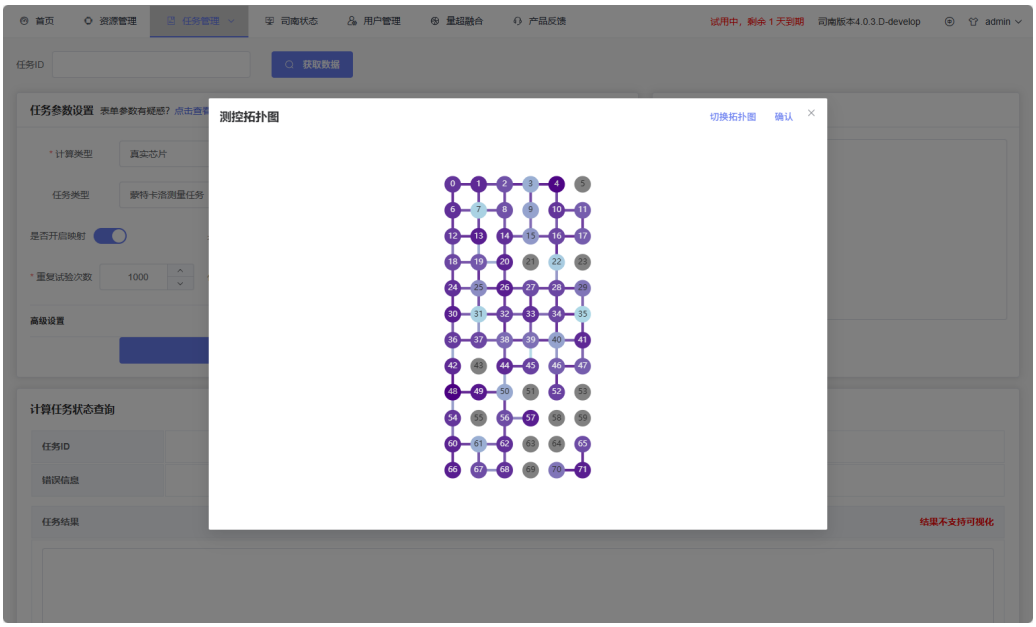


Figure 18: 用于选择指定量子比特块的芯片拓扑对话框。

  - **提交按钮：**将配置完成的任务提交至 PilotOS 系统服务。

- **量子程序区域（右侧）：**用于输入或上传 QProg。
  - **文本编辑器：**可直接粘贴 QProg
  - **文件上传：**支持 `.txt` 文件
- **任务结果区域（底部）：**
  查询或提交任务后，将显示以下信息：
  - 任务 ID
  - 错误信息（如有）
  - 编译前 / 编译后指令
  - 任务结果

  对支持可视化的任务，可以以柱状图形式展示结果。

操作流程：

1. **（可选）查询并恢复历史任务**输入任务 ID 并点击“查询”，系统自动恢复参数、QProg 及任务结果（如可用）。
2. **真实量子芯片任务**
   - 选择计算类型：真实量子芯片
   - 选择 Chip ID
   - （可选）通过拓扑对话框选择物理量子比特块
   - 配置测量类型、采样次数等参数

- 启用映射以避免逻辑与物理量子比特冲突
- （可选）配置高级设置
- 输入或上传 QProg
- 提交任务

3. **集群模拟任务**
   - 选择模拟器类型
   - 选择 Cluster ID
   - 填写模拟器相关参数
   - 输入或上传 QProg
   - （可选）设置优先级或描述
   - 提交任务

4.6.2 我的任务页面

功能概述：

我的任务页面用于显示并管理当前用户在 PilotOS 系统服务中提交的量子计算任务。可通过任务管理下的 **我的任务**子菜单访问。在该页面中，用户可以查看任务列表、搜索历史记录、按多种条件筛选任务，并跳转至任务详情页面或任务提交参数页面。

页面布局：

如图 19 所示，我的任务页面主要包含以下功能模块：

- **任务查询模块（顶部）：**提供多条件搜索功能，支持以下筛选项：
  - 计算类型（如全振幅模拟、噪声模拟、部分振幅模拟等）；
  - 后端类型（量子芯片或集群）；
  - 起止时间（任务创建时间范围）；
  - 任务描述（关键词搜索）。

  页面同时提供 **搜索**、**重置**和 **新增任务**按钮，便于快速操作。
- **任务列表模块（主区域）：**以表格形式展示任务信息，包括：
  - **任务 ID**：点击可进入任务详情页面；
  - **任务描述**
  - **任务类型**
  - **计算后端**
  - **任务状态**：如运行中、已完成、失败；
  - **开始时间**、**结束时间**及 **总时长**
  - **操作**：点击 **查看参数**跳转至任务提交页面查看提交配置。

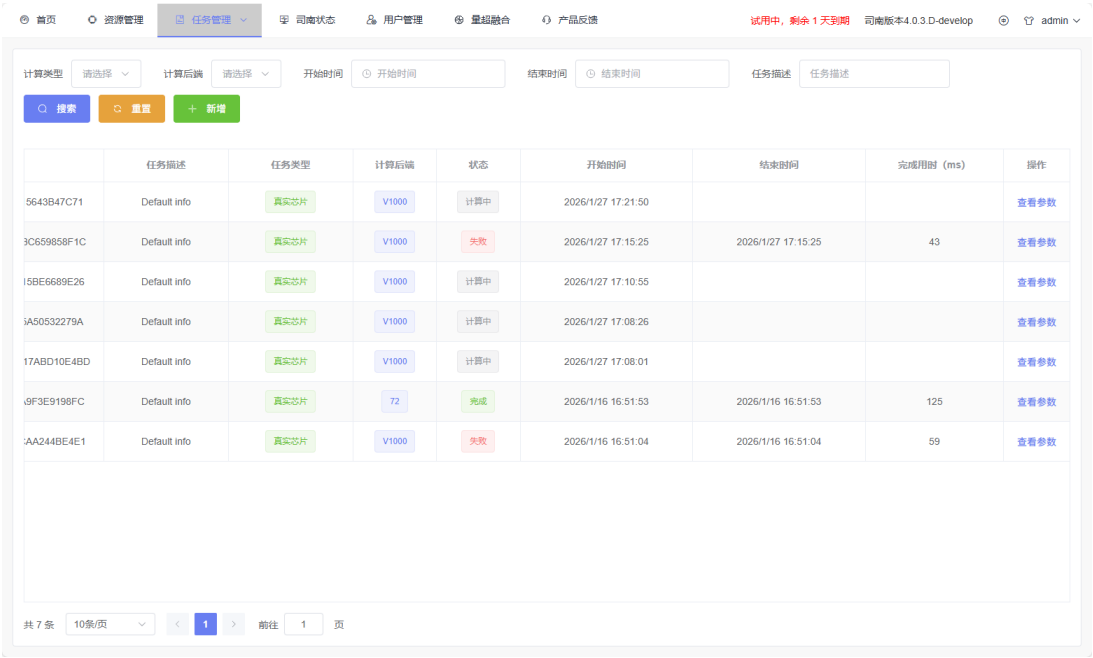


Figure 19: 我的任务页面，显示任务列表、搜索筛选及任务详情和参数链接。

4.6.3 任务详情页面

功能概述：

任务详情页面用于查看单个量子计算任务的详细信息及时间戳数据。可通过 **任务管理 → 任务详情**，或在我的任务列表中点击任务 ID 打开该页面。用户可按任务 ID 查询任务，并查看各处理阶段的时间戳、任务状态以及计算结果（如图 20 所示）。

页面布局：

任务详情页面包含以下功能模块：

- **任务查询模块（顶部）：** 输入任务 ID 并点击 **搜索**，即可获取任务详情及时间戳数据。
- **任务时间戳可视化模块（左侧）：** 以流程图形式展示任务在各模块的执行时间节点及状态。当无任务数据时显示空状态。
- **任务信息模块（右侧）：** 以卡片形式展示任务详细信息，包括：
  - 任务 ID（可点击跳转至任务提交页面查看参数）；
  - 错误信息（如有）；
  - 提交者；
  - 提交时间；
  - 计算类型（量子芯片或集群计算）；
  - 任务状态；
  - 总处理时间、Pilot 总时间及各阶段耗时（编译、排队、测量、后处理等）；
  - 任务结果。

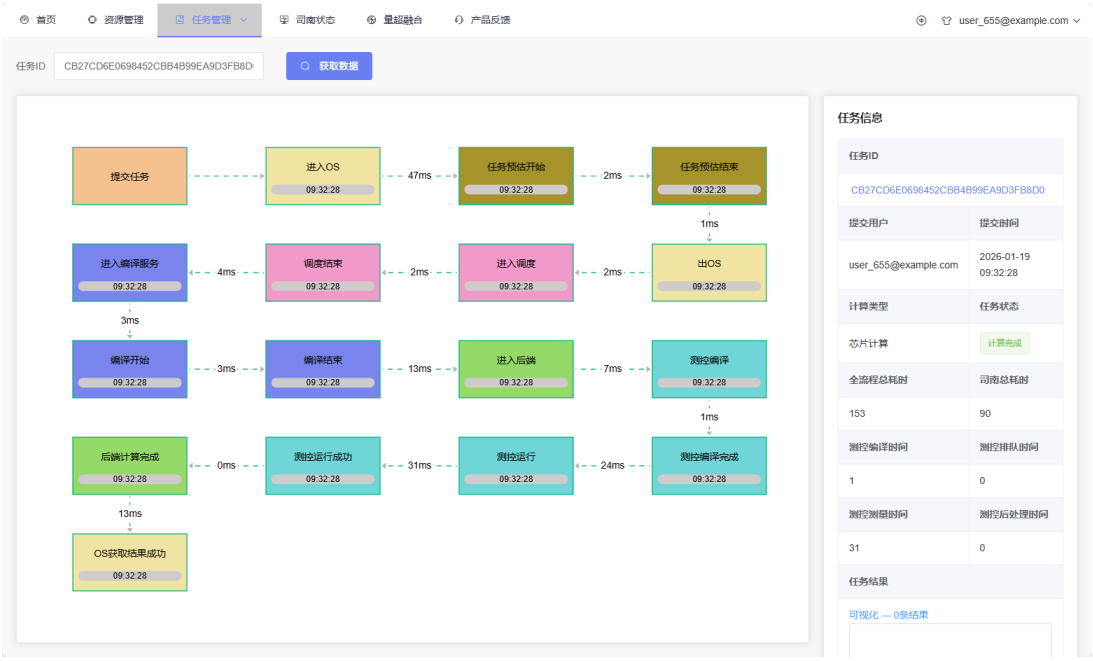


Figure 20: 任务详情页面概览：支持按任务 ID 查询，展示任务时间戳、模块执行耗时及计算结果。

4.6.4 实时任务状态页面

功能概述：

实时任务状态页面用于展示系统中当前排队和执行的任务。通过 **任务管理 → 实时任务状态**进入该页面。用户可监控活动任务、查看最新更新时间、检查任务详情，并支持按任务 ID 搜索，如图 21 所示。

页面布局：

- **任务查询模块：**
  - **任务 ID 查询：**输入任务 ID 定位特定任务；
  - **自动刷新控制：**
    - 开启刷新：持续更新任务状态；
    - 停止刷新：暂停自动更新。
  - **任务总数：**显示系统中排队任务总数。
- **任务列表表格：**以可排序表格形式展示任务，列包括：
  - **任务 ID**（点击跳转任务详情页面）；
  - **后端**；
  - **提交者**；
  - **提交时间**；
  - **任务状态**；

– **状态码**。

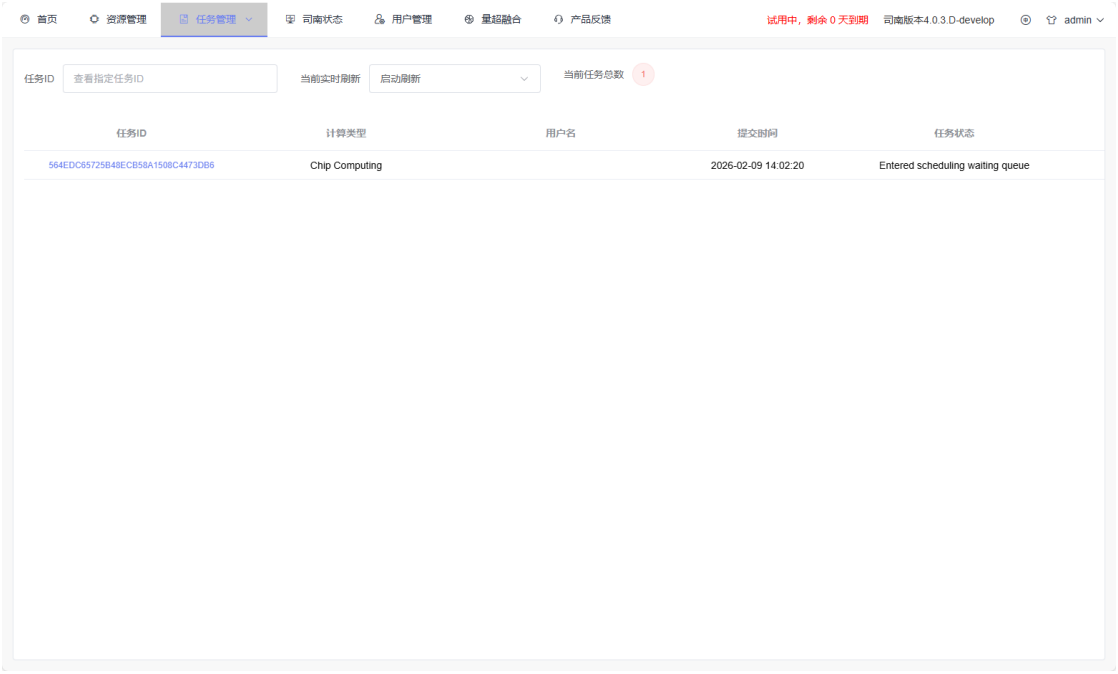


Figure 21: 实时任务状态页面，显示排队和执行任务、任务 ID 查询及自动刷新控制。

4.6.5 用户任务管理页面

功能概述：

用户任务管理页面用于显示并管理用户提交的排队任务。用户可查看自己在后端的任务队列，并取消自己的任务。管理员可管理所有用户的任务。

访问路径：**任务管理** → **用户任务管理**。页面概览见图 22。

页面布局：

- **顶部操作区域：**
  - **批量取消**：选择多个任务并点击 **取消任务**。
  - **队列大小**：显示当前任务队列总数。
  - **账户名称筛选**：管理员可按提交者账户过滤任务。
- **任务列表表格：**显示排队任务，列包括：
  - **选择**（用户只能选择自己提交的任务）；
  - **任务 ID**；
  - **计算类型**；
  - **任务状态**；
  - **提交者**；

– **提交时间**；

– **操作**：设置优先级 / 取消任务。

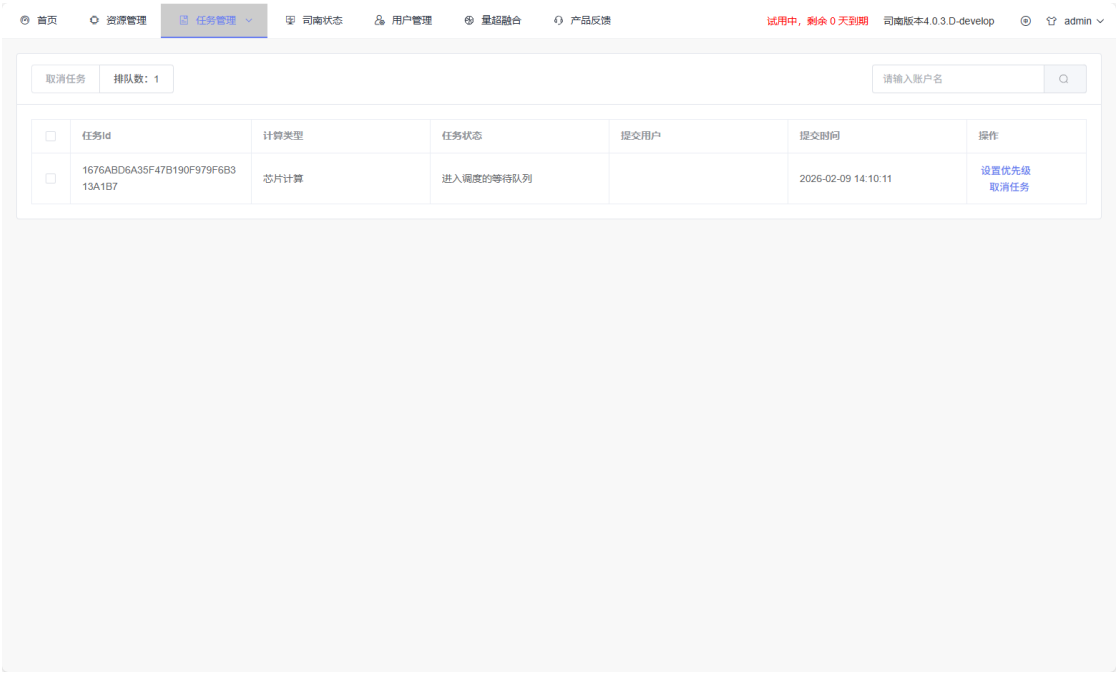


Figure 22: 用户任务管理页面

4.6.6 任务报表页面

功能概述：

任务报表页面用于查询、筛选并导出历史任务数据。用户可按时间范围、账户名、计算类型和任务状态进行筛选，并支持自定义显示列。页面支持任务列表展示、字段选择、任务 ID 复制及 CSV 导出。

访问路径：**任务管理** → **任务报表**。页面概览见图 23。

页面布局：

- **查询条件：**配置报表筛选：
  - **时间范围（必填）：**使用日期时间选择器选择开始和结束时间。
  - **账户名**：按提交者筛选任务。
  - **计算类型**：包括模拟器和真实量子芯片。
  - **任务状态**：
    * 计算中
    * 已完成
    * 失败

        * 排队中
        * 已取消
    - **操作**：
        * **搜索**：加载筛选后的任务；
        * **下载**：以 CSV 格式导出结果。
- **列选择：**可选择显示字段，包括：
    - 任务 ID
    - 账户名
    - 任务状态
    - QProg
    - 后端 ID
    - 任务类型
    - 配置信息
    - 任务结果
    - QST 结果 / QST 保真度
    - 各类时间指标
    - 开始时间 / 结束时间
    - 错误码 / 错误信息
    - SN / MC 时间戳
    - 任务描述
- **任务列表表格：**分页显示筛选后的任务：
    - **任务 ID**：完整显示，可点击跳转任务详情页面，并提供快速复制按钮；
    - **任务状态**：使用彩色标签显示；
    - **动态列**：表格列根据用户选择动态调整。

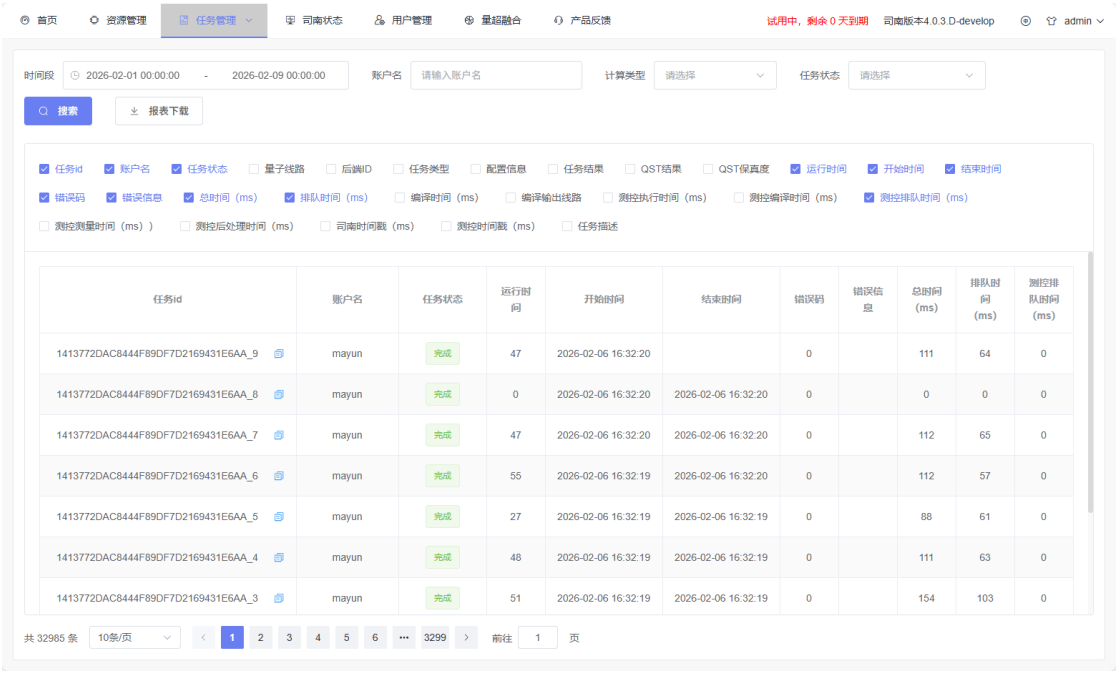


Figure 23: 任务报表页面

5 API 参考

以下是 PilotOS API 的详细介绍及代码示例。

- RESTful API 接口列表 (GET / POST / PUT / DELETE)
- 请求 URL、参数及示例
- 响应格式 (JSON)
- 错误代码对照表
- SDK 使用示例 (Python / cURL)

5.1 PilotOS 登录

URL: https://xx.com/management/pilotosmachinelogin

登录功能是使用 PilotOS OpenAPI 的前提条件，用于身份验证。登录后获取的 Token 允许用户执行后续操作（如提交任务）。登录时，必须使用用户的 API Key 进行身份验证。

请求方法： Post

必选参数：

- apiKey(str): PilotOS 使用的“数字 ID”或“专属密钥”。

返回值：

- errCode(uint): 错误代码。
- errInfo(str): 错误信息。

- token(str): 本次登录的用户身份标识符。

请求示例：

```
{
    "apiKey": "D13A8E5ADxxxxxxxxF36A4B940D18CEF"
}
```

响应示例：

```
{
    "errCode": 0,
    "errInfo": "",
    "token": "c3f7fe35-xxxx-xxxx-a178-070cc311d03b"
}
```

5.2 提交量子芯片计算任务

URL: https://xxx.com/task/realQuantum/run

向真实的物理量子芯片提交计算任务。注意：提交至量子芯片的任务**必须**包含测量（Measurement）操作。

请求方法： Post

必选参数：

- QProg(str): 量子线路。
- TaskType(int): 任务类型。

-1	未定义 (Undefined)
0	蒙特卡洛测量 (Monte Carlo Measurement)
2	量子态重构 (QST)
5	期望值计算 (Expectation)
10	分布式任务 (Distributed)

可选参数：

- Priority(uint): 优先级。0 表示普通任务（默认值），数值越大优先级越高。
- ChipID(str): 使用的芯片 ID。目前提供 72 和 102 位量子芯片，默认值为 72。
- Configuration(dict): 配置项。
  - shot(uint): 测量次数，默认 1000，范围 10-20000。
  - circuitOptimization(bool): 是否启用线路优化。true 为开启（默认），false 为关闭。

返回值：

- taskState(str): 任务状态。

0	未操作
1	已发送
2	运行中
3	已完成（测控系统已返回结果）
5	编译完成，未发送至测控系统
6	编译完成，已发送至测控系统
7	测控系统计算错误
8	运行时间超限
9	编译系统开始执行
10	结果正返回至编译系统
11	编译系统已接收任务并开始计算
12	编译系统已返回结果
22	编译系统正在处理并行任务
30	获取并行结算结果，发送至编译系统处理
31	已接收任务，准备处理
32	正在处理并行结果
33	并行结果处理完成
34	并行结果处理错误

- taskId(str): 已提交任务的任务 ID。

请求示例:

1. 普通测量任务:

```
{
  "token": "c3f7fe35-xxxx-xxxx-a178-070cc311d03b",
  "QMachineType": 5,
  "QProg": [
        "QINIT 6\nCREG 6\nH q[0]\nCNOT q[0],q[1]\nMEASURE q[0],c[0]\
           nMEASURE q[1],c[1]"
  ],
  "ChipID": "72",
  "TaskType": 0,
  "Priority": 0,
  "taskDescribe":"default",
  "Configuration": {
        "shot": 1000,
        "circuitOptimization": True,
        "use_cz_pattern": False,
        "amendFlag": 0,
```

```
        "mappingFlag": True,
        "specified_block": [],
        "PointLabel": 1
    }
}
```

2. 量子态层析任务:

```
{
    "token": "c3f7fe35-xxxx-xxxx-a178-070cc311d03b",
    "QMachineType": 5,
    "QProg": [
        "QINIT 6\nCREG 6\nH q[0]\nCNOT q[0],q[1]\nMEASURE q[0],c[0]\
           nMEASURE q[1],c[1]"
    ],
    "ChipID": "72",
    "TaskType": 2,
    "Priority": 0,
    "Configuration": {
        "shot": 1000,
        "circuitOptimization": True,
        "use_cz_pattern": False,
        "amendFlag": 0,
        "mappingFlag": True,
        "specified_block": [],
        "PointLabel": 1
    }
}
```

3. 分布式任务:

```
{
    "QMachineType": 5,
    "ChipID": "72",
    "TaskType": 10,
    "token": "c3f7fe35-xxxx-xxxx-a178-070cc311d03b",
     “QProg” : [
         “QINIT 6\nCREG 6\nRY(q[0], 0.162461319481123)\nRY(q[1],
           1.13411443752956)\nRY(q[2], 0.349219035305167)\nCNOT(q[0],
            q[1])\nRY(q[0], -1.30314609114098)\nRY(q[1],
           -0.000403011347911202)\nCNOT(q[1], q[2])\nRY(q[1],
           -0.0668867624864373)\nRY(q[2], 0.751632608892654)\nCNOT(q
           [2], q[0])\nRY(q[2], 0.943957761048972) \nRY(q[0],
```

```
            0.877382733682492)\nCNOT(q[2], q[0])\nCNOT(q[1], q[2])\
            nCNOT(q[0], q[1])"
    ],
    "Configuration": {
        "shot": 1000,
        "mappingFlag": true,
        "construction_type" : 0,
        "max_supported_qubits" : 2
    }
}
```

4. 求期望任务:

```
{
    "token": "c3f7fe35-xxxx-xxxx-a178-070cc311d03b",
    "QMachineType": 5,
    "QProg": ["QINIT 6\nCREG 6\nH q[0]\nCNOT q[0],q[1]"],
    "ChipID": "72",
    "TaskType": 5,
    "Configuration": {
        "shot": 1000,
        "circuitOptimization": True,
        "hamiltonian": "\"\": -1.0534210769165204,\"Z0\":
           0.394844363355904,\"Z1\": -0.39484436335590417,\"Z0 Z1\":
           -0.01124615715082114,\"X0 X1\": 0.18121046201519694"
    }
}
```

响应示例:

```
{
    "taskId": "098D12FE69184646BF16DDF53930C569",
    "taskState": 1
}
```

5.3 查询任务

URL: https://xxx.com/management/query/taskinfo

查询此前提交的任务，返回其执行状态及相关信息。若任务已完成，则同时返回任务结果。

请求方法： Post

必选参数：

- taskid(str): 任务 ID
- token(str): 身份 Token

返回值：

- taskId(str): 任务 ID
- TaskType(int): 任务类型
- taskState(uint): 任务状态
- qProgLength(uint): 量子线路数量
- qProg(arr[str]): 量子线路详情
- taskResult(str): 量子任务执行结果

请求示例:

```
{
    "taskid":"098D12FE69184646BF16DDF53930C569",
    "token":"c3f7fe35-xxxx-xxxx-a178-070cc311d03b"
}
```

响应示例:

```
{
    "taskId": "098D12FE69184646BF16DDF53930C569",
    "TaskType": 1,
    "taskState": 3,
    "qProgLength": 1,
    "without_compensate_prog": "",
    "qProg": [
        "QINIT 4\nCREG 4\nH q[0]\nMEASURE q[0],c[0]"
    ],
    "taskResult": "[{\"key\":[\"0x0\", \"0x1\"],\"value\":[0.52,
       0.48]}]"
}
```

5.4 获取队列信息

URL: https://xxx.com/system/queryBackendQueueTask

获取 PilotOS 芯片后端的任务队列信息。

请求方法： Post

必选参数：

- backend(str): 芯片 ID。
- maxQueryTaskSize(uint): 最大查询任务数量。

返回值：

- msg_type(str): 消息类型。
- backend(uint): 后端 ID。
- is_ok(bool): 查询是否成功。
- queue_task_size(uint): 找到的任务总数。
- myTask(arr[str]): 检索到的任务 ID 集合。
- userNames(arr[str]): 对应任务的用户名称集合。

请求示例:

```
{
    "backend":"72",
    "maxQueryTaskSize":500
}
```

响应示例:

```
{
    "msg_type":"schdule_queue_resp",
    "backend":"72",
    "is_ok":true,
    "queue_task_size":2,
    "myTask":["FCECF3B04053402895C2485E67FAEB80","
       B859CD543212488DB8582B45E940CFC9"],
    "userNames":["user1","user2"]
}
```

5.5 获取芯片信息

URL: https://xxx.com/management/getchipconfig

获取芯片的详细配置信息。

请求方法： Post

必选参数：

- ChipID(str): 芯片 ID。
- Label(uint): 不同标签下的芯片具有不同特性（如保真度）。

返回值：

- QubitParams(json obj): 每个比特的保真度、相干时间等参数。
- NoiseModel(json obj): 真实环境下的芯片噪声模型。
- CompensateAngle(json obj): 每对比特之间 CZ 门的串扰补偿。
- AvailableQubits(json obj): 芯片上的可用比特。

请求示例:

```
{
    "ChipID":"72",
    "Label":1
}
```

响应示例:

```
{
    "QubitParams":
    [
        {
            "qubitId":"45",
            "FidelityMat":[[0.9556,0.1218],[0.0444,0.8782]],
            ...
        },
        ...
    ],
    "NoiseModel":[
        {"modelType":"noise_model_single},
        ...
    ],
    "CompensateAngle":
    {
        "CZ(45,46)":{"45":2.367866260192345…},
        ...
    },
    "AvailableQubits":[45,46,48,52,53,54],
    ...
}
```

5.6 提交集群模拟计算任务

URL: https://xx.com/task/run

提交计算任务到集群进行模拟计算。

请求方法: Post

参数说明:

参数描述	参数命名	类型	备注	是否为空
量子程序	QProg	String		不可空
量子程序长度	QProgLength	uint		可空
配置信息	Configuration	json	不同任务类型的配置信息	不可空
任务类型	TaskType	uint	参考任务类型定义	可空
虚拟机类型	QMachineType	String	0: 全振幅, 1: 噪声, 2: 部分振幅, 3: 单振幅, 4:MPS	不可空
VIP 类型	Priority/VIP	uint/string	0 为普通任务，数值越大优先级越高	可空
集群 ID	ClusterID	string	为空时系统自动分配	可空
回调地址	callbackAddr	String		可空
唯一标识码	Token	string	用户标识	可空

5.6.1 Configuration 信息说明及请求示例

（1）全振幅虚拟机（QMachineType: 0）

Configuration 参数：

- measure_type (uint): 1-Measure 测量; 2-PMeasure; 3-哈密顿量（不可空）。
- shot (uint): 测量次数（不可空）。
- qubits (uint 数组): PMeasure 和哈密顿量时必须指定。
- hamiltonian (数组/string): 哈密顿量参数（measure_type 为 3 时不可空）。
- hamiltonian_param (double): 哈密顿量系数（数组格式时）。
- pauli_type (str 数组): 泡利算符类型（数组格式时）。
- pauli_param (double 数组): 算符作用比特（数组格式时）。

请求示例 (Measure):

```
{
  "QMachineType":"0",
  "QProg":"QINIT 4\r\nCREG 4\r\nTOFFOLI q[3],q[0],q[1]\r\nMEASURE q
     [1],c[1]",
  "QProgLength":58,
  "Configuration":{
    "measure_type": 1,
    "shot": 1000
  }
}
```

请求示例 (PMeasure):

```
{
  "QMachineType":"0",
  "QProg":"QINIT 4\r\nCREG 4\r\nH q[0]\r\nCNOT q[0],q[1]...",
  "QProgLength":71,
  "Configuration":{
```

```
    "measure_type":2,
    "qubits":[0,1,2,3]
  }
}
```

请求示例 (哈密顿量):

```
{
  "QMachineType":"0",
  "Configuration":{
    "qubits":[0,1,2,3,4,5],
    "hamiltonian":[
        {"pauli_type": ["X", "Y"], "pauli_param": [0, 1], "
           hamiltonian_param": 0.2},
        {"pauli_type": ["Y", "X"], "pauli_param": [2, 3], "
           hamiltonian_param": 0.8}
    ],
    "measure_type":3
  }
}
```

（2）部分振幅虚拟机 (QMachineType: 2)

Configuration 参数:

- Amplitude (string 数组): 十进制索引数组（不可空）。

请求示例:

```
{
  "QMachineType":"2",
  "Configuration":{
    "Amplitude":["0"]
  }
}
```

（3）单振幅虚拟机 (QMachineType: 3)

Configuration 参数:

- Amplitude (string): 十进制索引（不可空）。

请求示例:

```
{
  "QMachineType":"3",
  "Configuration":{
    "Amplitude":"0"
```

```
  }
}
```

(4) 噪声虚拟机 (QMachineType: 1)

Configuration 参数:

- measure_type (uint): 必须为 1。
- shot (uint): 测量次数。
- noisemodel.single_gate (数组): 单门操作数据 [类型 str, 参数 double]。
- noisemodel.double_gate (数组): 双门操作数据 [类型 str, 参数 double]。
- mode (uint): 噪声模型类型。

请求示例:

```
{
    "QMachineType":"1",
    "Configuration":{
        "mode":1,
        "measure_type":1,
        "noisemodel":{
            "single_gate":["DAMPING_KRAUS_OPERATOR",0.100000],
            "double_gate":["DAMPING_KRAUS_OPERATOR",0.200000]
        },
        "shot":1000
    }
}
```

(5) MPS 虚拟机 (QMachineType: 4)

Configuration 参数:

- measure_type (uint): 1-Measure; 2-PMeasure。
- shot (uint): 测量次数。
- qubits (int 数组): PMeasure 时不可空。

返回参数说明:

- taskState (string): 任务状态。
- taskId (string): 任务 ID。

5.7 集群模拟计算任务状态查询

URL: https://xx.com//task/query

请求方法: Post

参数:

- taskId (String): 任务 ID（不可空）。

请求示例:

```
{"taskId":"5EB2161414632304351f8a9C5BBe124ef1b31d71ced321fA92"}
```

5.8 获取指定后端的任务排队信息

URL: https://xx.com/system/queryBackendQueueTask

请求方法： Post

参数:

- backend (string): 芯片 ID（不可空）。
- maxQueryTaskSize (uint32): 最大返回任务数目（不可空）。
- token (string): 唯一标识码（可空）。

返回示例:

```
{
  "msg_type":"schdule_queue_resp",
  "backend":"72",
  "is_ok":true,
  "queue_task_size":2,
  "queue_task":["task_id_1","task_id_2"],
  "myTask":["FCECF3B04...", "B859CD5..."],
  "userNames":["yebin","yebin"]
}
```

5.9 获取芯片校准时间

URL: http://xx.com/system/queryChipUpdateTime

请求方法： Post

参数:

- backend (string): 后端 ID（不可空）。

请求示例:

```
{"backend":"72"}
```

返回示例:

```
{
    "backend":"72",
    "chipLastUpdateTime": {
        "qubit":[43,45...],
```

```
        "timeStamp":[1687243363189...]
    },
    "errCode":0
}
```

5.10 暂停芯片调度服务

URL: http://xx.com/system/pauseChipSchedule

请求方法： Post

参数：

- ChipID (string): 芯片名称（不可空）。
- isPause (bool): true 开启调度，false 暂停调度（可空，默认为 true）。

请求示例:

```
{
    "isPause":false,
     "ChipID":"72"
}
```

返回示例:

```
{
    "scheduleState":false,
     "errCode":,
     "errInfo":""
}
```

5.11 获取芯片调度服务状态

URL: http://xx.com/system/queryChipScheduleState

请求方法： Post

参数：

- ChipID (string 数组): 芯片 ID 数组（不可空）。

请求示例:

```
{
    "ChipID":["1","72"]
}
```

返回示例:

```
{
    "ScheduleState" :[
      { "chipID":"1", "status":true },
      { "chipID":"72", "status":true }
    ],
    "errCode":0,
    "errInfo":""
}
```

5.12 手动进行 M3 采样

URL: http://xx.com/system/manualSampleM3

请求方法： Post

参数：

- Token (String): 用户 Token。
- ChipID (string): 芯片 ID。
- PointLabel (Uint): 工作点标签。

请求示例:

```
{
    "Token":"3A9D2FD3ECCB4B0C",
     "ChipID":"72",
     "PointLabel":1
}
```

返回示例:

```
{
    "errCode":0,
    "errInfo":""
}
```

5.13 Session 机制任务管理

用户可以申请一个会话，以会话为单位提交任务，拥有更高优先级。

5.13.1 请求会话

URL: http://xx.com//session/apply

参数：

- LifeTime (int): 会话有效时间（分）。

- Token (String): 用户 Token。

请求示例:

```
{
    "LifeTime" : 30,
    "Token" : "D13A8E5AD804478FBF36A4B940D18CEF"
}
```

返回示例:

```
{ "SessionId" : "6D94C74A...", "errCode": 0 }
```

5.13.2 在会话中提交任务

URL: http://xx.com//session/append_task

参数: 与“提交计算任务到物理量子芯片”一致，需额外增加:

- SessionId (string): 会话 ID（不可空）。

请求示例:

```
{
    "QMachineType": "5",
     "ChipID":"72",
     "TaskType":3,
     "QProg":["QINIT 4\r\nCREG 4\r\nH q[3]..."],
     "QProgLength":  2,
     "Configuration":{
             ...
        },
     "taskDescribe": "default",
     "SessionId": "6D94C74A73..."
}
```

返回示例:

```
{ "taskState":"1","taskId":"6D94C74A73A544FD9C445D51E7436191_0" }
```

5.13.3 关闭会话

URL: http://xx.com//session/release

参数:

- SessionId (string): 会话 ID。
- Token (string): Token。

请求示例:

```
{
    "SessionId" : "6D94C74A73A544FD9C445D51E7436191",
    "Token" : "D13A8E5AD804478FBF36A4B940D18CEF"
}
```

返回示例:

```
{
    "SessionId": "6D94C74A73A544FD9C445D51E7436191",
    "isSuccess": false,
    "errCode": 0,
    "errInfo": "Task is running."
}
```

5.14 获取芯片最优量子比特块

URL: http://xx.com//management/query/best_qubit_blocks

请求方法: Post

参数:

- ChipID (string): 芯片 ID。
- Label (uint32): 标签类型。
- QubitNum (uint32): 比特数。
- QubitBlockNum (uint32): 比特块数。

请求示例:

```
{
    "ChipID" : "72",
    "Label" : 1,
    "QubitNum" : 4,
    "QubitBlockNum" : 1
}
```

返回示例:

```
{
  "ChipID":"72",
  "QubitBlocks":[[1,2,3,4]],
  "errCode":0
}
```

5.15 获取任务的优先级

URL: http://xx.com///task/query_task_priority

请求方法： Post

参数：

- TaskID (string): 任务 ID。
- Token (string): 用户标识。

请求示例:

```
{
    "TaskID" : "ABC",
    "Token" : "asedsdfsdg",
}
```

返回示例:

```
{ "TaskID":"ABC", "Priority":10 }
```

6 消息推送

系统会向配置的地址推送关键事件消息。以下是各消息类型的详细说明及推送示例。

6.1 芯片校准开始信息

推送地址: http://xxx:31733/v5/oqcs/task/updateChipOnlineStatus.json

参数说明:（若没有校准比特信息和 CZ 比特对信息则不推送）

- msg_type (string): 固定为"chip_calibration_start"。
- chip_id (string): 芯片 ID。
- calibration_qubits (string array): 校准比特。
- calibration_couplers (string array): 耦合器。
- calibration_pairs (string array): CZ 比特对。
- calibration_discriminators (string array): 判据。

推送示例:

```
{
  "msg_type": "chip_calibration_start",
  "chip_id": "72",
  "calibration_qubits": ["q0", "q1", "q2", "q3"],
  "calibration_couplers": ["c0-1", "c1-2", "c2-3"],
  "calibration_pairs": ["q0q1", "q2q3"],
  "calibration_discriminators": ["q0_01.bin"]
}
```

6.2 芯片校准结束信息

推送地址: http://xxx:31733/v5/oqcs/task//updateCalculateResource.json

参数说明:

- chipId (string): 芯片 ID。
- lastCalibrationTime (string): 最后校准时间戳。
- lastCalibrationBit (string array): 校准比特列表。
- lastCalibrationBitPair (string array): 校准比特对列表。

推送示例:

```
[
    {
        "chipId": "72",
```

```
        "lastCalibrationTime": "1731576994075",
        "lastCalibrationBit": [],
        "lastCalibrationBitPair": []
    }
]
```

6.3 芯片新配置信息

推送地址: http://xxx:31733/v5/oqcs/task/updateChipConfig.json

参数说明:

- msg_type (string): 固定为”chip_update”。
- chip_id (string): 芯片 ID。
- new_chip_config (json string): 新的芯片配置 JSON 字符串。

推送示例:

```
{
  "msg_type": "chip_update",
  "chip_id": "72",
  "new_chip_config": "{
    \"QuantumChipArch\": {
        \"QubitCount\": 72,
        \"HighFrequencyQubit\": [
            0,
            4,
            9,
            13,
            16,
            21,
            24
        ]
    }
  }"
}
```

6.4 芯片人工维护开始信息

推送地址: http://xxx:31733/v5/oqcs/task/updateChipOnlineStatus.json

参数说明:

- msg_type (string): 固定为”chip_manual_maintenance_start”。

- chip_id (string): 芯片 ID。

推送示例:

```
{
  "msg_type": "chip_manual_maintenance_start",
  "chip_id": "72"
}
```

6.5 芯片人工维护结束信息

推送地址: http://xxx:31733/v5/oqcs/task/updateChipOnlineStatus.json

参数说明:

- msg_type (string): 固定为"chip_manual_maintenance_end"。
- chip_id (string): 芯片 ID。

推送示例:

```
{
  "msg_type": "chip_manual_maintenance_end",
  "chip_id": "72"
}
```

7 类型定义与错误码

7.1 任务类型

类型码	说明
-1	未知任务类型
0	蒙特卡洛测量任务
1	PMeasure 测量任务
2	QST 任务
3	QST 密度矩阵计算任务
4	保真度计算任务
5	求期望任务
8	噪声学习任务
9	读取误差缓解任务
10	量子分布式计算任务
11	保真度测量任务

7.2 计算后端类型

类型码	说明
0	全振幅模拟
1	噪声模拟
2	部分振幅模拟
3	单振幅模拟
4	MPS 模拟
5	真实芯片计算

7.3 任务状态类型

状态码	说明
0	未知状态
1	系统接收到计算任务
2	正在被执行
3	计算成功
4	计算失败
5	排队中
35	任务已被取消

7.4 错误码 (ErrorCode)

错误码	说明
0	无错误
1	数据库错误
2	OriginIR 语法错误
3	Json 解析错误
4	后端计算错误
5	系统任务队列满
6	Qubit 数目超出限制
7	不支持的后端类型
8	量子程序时序超出
9	任务类型错误
10	虚拟机初始化失败
11	编译失败
12	任务预估错误
13	元数据配置错误
14	保真度密度矩阵错误
15	QST 程序错误
16	空的量子程序
17	Qubit 数目错误，和目标逻辑门不匹配
18	量子芯片 Qubit 拓扑结构错误
19	量子芯片计算任务程序错误
20	重复的 Measure 操作
21	数据库操作错误

错误码	说明
22	系统任务状态信息满
23	Backend 模块发送任务到后端时通信失败
24	集群计算错误
25	调度错误，芯片拓扑结构不支持线路运行
26	任务配置错误
27	未知的应用 ID
28	未知的任务 ID
29	解析子任务结果错误
30	系统调用超时
31	任务已终止
32	URL 地址错误
33	参数错误
34	量子线路长度错误
35	芯片离线状态，无法设置真实计算模式
36	未知错误
37	登录密码错误
38	用户名已注册
39	Token 验证失败
40	任务查询失败
41	用户查询失败
42	权限不足
43	API Key 错误
>1400	测控错误（标记硬件错误）

8 运维与管理

8.1 系统维护

服务启动/停止命令

方法 1：用户可通过管理页面启停司南系统，详细操作方法参考“ 4.5 司南状态页面”中的相关说明。

方法 2：重新拉起镜像，详细操作方法参考“2.5 服务启动与验证”章节内容。

数据库备份与恢复

司南服务所使用的 mysql 和 MongoDB 数据库都由用户自行配置，用户可根据自身需求选择合适的数据库备份方案。

用户权限管理

详细操作方法参考“ 4.2.3用户管理”中的相关说明。

系统升级流程

详细操作方法参考“ 4.5 司南状态页面”中的相关说明。

8.2 故障排除

常见问题及其解决方案列表。

问题列表：

- **任务停留在”Sent”（已发送）状态**
  可能原因：与测控系统断开连接。
  解决方案：检查网络连接并重启测控服务。
- **数据库连接失败**
  可能原因： Redis 未运行。
  解决方案：执行 systemctl start redis。
- **编译失败**
  可能原因：语法错误。
  解决方案：检查提交任务的线路格式。
- **页面加载缓慢**
  可能原因：缓存堆积。
  解决方案：清除浏览器缓存或服务器端缓存。